PYTHON

面试宝典

100 道核心真题 · 专业题解版

语言语义	并发与 CPython	工程与系统设计

从“会写 Python”到“能解释 Python、能设计 Python 系统”

题目 → 考点 → 面试回答 → 原理拆解 → 代码实验 → 易错点 → 连续追问 → 举一反三

适用：Python 后端 / 算法工程 / 数据工程 / AI Infra / 通用软件工程面试

版本基线：Python 3.14（版本敏感内容单独标注）

整理原则：公开高频题型 + 官方语言/标准库资料；不声称来自不可验证的企业内部题库。

如何使用本书

这不是“背答案”的 100 题。每一道题都按真实面试的推进方式组织：先给 30-60 秒可口述的核心回答，再用原理、代码、陷阱和追问把知识扩展到可迁移层。

推荐刷题节奏：

- 第一轮：只读“面试标准回答”和“易错点”，建立覆盖面。
- 第二轮：关闭答案，自己口述 60 秒，并运行代码实验。
- 第三轮：只看“面试官追问”，训练连续 3-5 层追问能力。
- 冲刺轮：使用本书末尾速查表与 20 道 S 级题复盘。

难度与频率标记

标记	含义	建议
★-★★	基础/常规	必须做到准确、快速
★★★	中级机制	要能解释原理和边界
★★★★-★★★★★	高级/实现/系统	用于中高级岗位拉开差距
频率：极高/高	公开面经长期反复出现	优先背诵并口述
版本敏感	CPython/Python 版本会改变结论	回答必须给版本边界

目录与知识地图

01 Python 基础语义与对象模型 | Q01-Q10

建立 Python 的正确心智模型：名字绑定、对象身份、可变性、作用域和调用语义。

02 核心容器与哈希表 | Q11-Q20

从 list/tuple/dict/set 的 API 进入复杂度、哈希契约、拷贝语义与工程选择。

03 函数、闭包与装饰器 | Q21-Q30

把函数当一等对象理解，掌握闭包、late binding、装饰器、缓存与调用签名。

04 OOP 与 Python Data Model | Q31-Q40

理解属性访问、descriptor、MRO、super、metaclass 与 __slots__ 等高级机制。

05 迭代器、生成器与资源管理 | Q41-Q50

掌握惰性计算、迭代协议、generator delegation 与 context manager。

06 CPython、内存、GC 与 Import | Q51-Q60

区分语言规范与 CPython 实现，理解引用计数、循环 GC、分配器与导入系统。

07 线程、进程、GIL 与多核并行 | Q61-Q70

掌握经典 GIL 模型，并覆盖 Python 3.14 的 free-threaded 与 InterpreterPoolExecutor。

08 asyncio 与结构化并发 | Q71-Q80

从 event loop 到 TaskGroup、取消传播、限流、背压与生产级 worker。

09 Typing、测试、打包与工程质量 | Q81-Q90

掌握现代类型系统、dataclass、测试边界、依赖隔离与 pyproject.toml。

10 性能优化、Web 后端与系统设计 | Q91-Q100

把语言知识落到 profiling、内存、流处理、缓存和高并发 API 的综合场景。

全局知识链

名字绑定 → 可变性/哈希 → 函数与闭包 → descriptor/MRO → 迭代与资源 → CPython/GC → GIL/并行 → asyncio/结构化并发 → typing/测试 → 性能与系统设计

CHAPTER 01

Python 基础语义与对象模型

建立 Python 的正确心智模型：名字绑定、对象身份、可变性、作用域和调用语义。

本章目标：能够用对象绑定模型解释参数、可变性、作用域和常见“反直觉”代码。

本章题目

Q01 Python 是解释型语言还是编译型语言？ 难度 ★★ · 频率 高 · CPython, bytecode, dis

Q02 is 和 == 有什么区别？ 难度 ★ · 频率 高 · identity, equality, __eq__

Q03 Python 变量保存的是值还是对象？ 难度 ★★ · 频率 高 · binding, object model, reference

Q04 Python 是 pass-by-value 还是 pass-by-reference？ 难度 ★★ · 频率 高 · arguments, object sharing, binding

Q05 mutable 和 immutable 的本质区别是什么？ 难度 ★★ · 频率 高 · mutability, hashability, aliasing

Q06 None 到底是什么？为什么推荐 is None？ 难度 ★ · 频率 高 · None, singleton, identity

Q07 Python 的 truthy / falsy 规则是什么？ 难度 ★★ · 频率 高 · bool, __bool__, __len__

Q08 and / or 为什么不一定返回 bool？ 难度 ★★ · 频率 中 · short-circuit, truthiness, expression

Q09 为什么 mutable default argument 是经典陷阱？ 难度 ★★ · 频率 极高 · default args, function object, state

Q10 global 和 nonlocal 有什么区别？LEGB 如何工作？ 难度 ★★ · 频率 高 · scope, LEGB, closure

Q01 Python 是解释型语言还是编译型语言?

难度：★★ 频率：高 关键词：CPython, bytecode, dis

30-60 秒面试标准回答

不要把“Python”与“CPython 的执行方式”混为一谈。CPython 通常先把源码编译为字节码，再由解释器执行；PyPy、MicroPython 等实现可以采用不同执行策略。

原理拆解

- 语言规范描述语义，不强制唯一实现方式。面试时先区分 Python 语言与 CPython 实现。
- CPython 的模块代码会经历解析、AST、编译为 code object/bytecode、解释执行等阶段；.pyc 主要是字节码缓存。
- 字节码不是稳定的跨版本 ABI。性能讨论还要区分解释器优化、JIT、C 扩展和 native code。

最小可运行实验

```
import dis

def add(a, b):
    return a + b

dis.dis(add)
```

易错点 / GOTCHAS

- 只回答“Python 是解释型语言”。
- 把 .pyc 说成机器码或可跨 Python 版本稳定运行的二进制。

面试官连续追问

1. 什么是 code object?
2. 为什么第一次 import 往往比后续 import 更慢?
3. PyPy 的 JIT 与 CPython 的执行模型有什么差异?

> **举一反三** 把“语言语义”和“实现细节”分开，是后续讨论 GIL、GC、内存分配器的共同方法。

Q02 is 和 == 有什么区别?

难度：★ 频率：高 关键词：identity, equality, __eq__

30-60 秒面试标准回答

`is` 比较对象身份；`==` 比较值相等语义，通常委托给 `__eq__`。检查 `None` 时优先使用 `is None`。

原理拆解

- identity 是“是否同一个对象”，value equality 是“语义上是否相等”。
- 整数/字符串驻留属于实现优化，不能据此用 is 判断数值或字符串内容。
- 自定义类可以重载 __eq__，所以 `x == None` 可能触发用户代码；`x is None` 不会。

最小可运行实验

```
a = [1, 2]
b = [1, 2]
print(a == b)  # True
print(a is b)  # False
print(a is None)
```

易错点 / GOTCHAS

- 用 `is` 比较字符串或数字内容。
- 误以为 `id(a) == id(b)` 与 `a is b` 在所有生命周期场景都可互换。

面试官连续追问

1. `__eq__` 返回 `NotImplemented` 时发生什么?
2. 为什么 `None` 适合 identity comparison?

> **举一反三** 把 identity/equality 分开后，dict/set 的 hash 契约会更容易理解。

Q03 Python 变量保存的是值还是对象?

难度：★★ 频率：高 关键词：binding, object model, reference

30-60 秒面试标准回答

Python 变量更准确地说是“名字到对象的绑定”。赋值语句通常改变绑定，而不是把某个传统意义上的“盒子里的值”复制进去。

原理拆解

- `a = b` 让两个名字绑定到同一个对象，不会自动复制对象。
- 可变对象被原地修改时，所有持有该对象引用的代码都能观察到变化；重新绑定某个名字则不会影响其它名字。
- 理解 binding model 是参数传递、闭包、浅拷贝、类属性共享等题目的基础。

最小可运行实验

```
a = [1, 2]
b = a
b.append(3)
print(a)  # [1, 2, 3]

b = [9]
print(a)  # 仍是 [1, 2, 3]
```

易错点 / GOTCHAS

- 把 `a = b` 描述为“复制 b 的值”。
- 看到外部对象变化就笼统说“Python 是引用传递”。

面试官连续追问

1. 赋值与切片复制有什么区别?
2. 不可变对象为什么看起来像“按值传递”?

> **举一反三** 画“名字 -> 对象”的引用图，是解释 Python 语义最可靠的方法之一。

Q04 Python 是 pass-by-value 还是 pass-by-reference?

难度：★★ 频率：高 关键词：arguments, object sharing, binding

30-60 秒面试标准回答

更准确的术语是 call by object sharing / call by assignment：函数形参会绑定到调用方传入的对象；修改该对象可能被外部观察到，而重新绑定形参不会修改调用方名字。

原理拆解

- 调用 `f(a)` 时，不是把变量 a 本身传进去，而是把 a 当前绑定的对象交给形参建立新的绑定。
- 对 list/dict 进行 append/update 是修改共享对象；`x = new_obj` 是重新绑定局部名字。
- 是否产生外部副作用取决于对象可变性、函数是否原地修改，以及外部是否还持有引用。

最小可运行实验

```
def mutate(x):
    x.append(3)

def rebind(x):
    x = [100]

a = [1, 2]
mutate(a)
rebind(a)
print(a)  # [1, 2, 3]
```

易错点 / GOTCHAS

- 用“引用传递”解释所有现象。
- 忽略可变对象别名导致的副作用。

面试官连续追问

1. 如何设计无副作用 API?
2. 为什么 tuple 中包含 list 时仍可能观察到内部变化?

> **举一反三** 工程上，文档应明确函数是“返回新对象”还是“原地修改”，避免调用者猜测。

Q05 mutable 和 immutable 的本质区别是什么?

难度：★★ 频率：高 关键词：mutability, hashability, aliasing

30-60 秒面试标准回答

可变性指对象创建后其可观察状态能否原地改变。不可变并不意味着“对象引用链上的所有东西都不可变”。

原理拆解

- int/str/tuple 等常见类型是不可变对象；list/dict/set 是可变对象。
- tuple 只保证 tuple 自身保存的元素引用序列不被替换；如果元素指向可变对象，该对象仍然可以变化。
- hashable 常与不可变性相关，但它是独立协议：核心是哈希值在对象参与哈希表期间必须稳定，并遵守 equality-hash 契约。

最小可运行实验

```
x = ([1, 2], 3)
x[0].append(4)
print(x)  # ([1, 2, 4], 3)
```

易错点 / GOTCHAS

- 把 tuple 说成“内部所有内容永远不会变化”。
- 把 immutable 与 hashable 完全画等号。

面试官连续追问

1. frozenset 为什么可哈希?
2. 自定义不可变类如何设计?

> **举一反三** 不可变数据更容易并发共享、缓存和推理，但工程上仍需考虑对象内部引用的传递。

Q06 None 到底是什么？为什么推荐 is None?

难度：★ 频率：高 关键词：None, singleton, identity

30-60 秒面试标准回答

`None` 是 `NoneType` 的唯一常用实例，用来表示“没有值/缺失结果”等语义。检查它时推荐 identity comparison。

原理拆解

- `None` 与空字符串、0、False 的业务语义不同，虽然这些值都可能是 falsy。
- `x is None` 不会触发用户自定义相等逻辑，语义也直接表达“是否是这个单例对象”。
- API 设计中要区分“未提供参数”“显式传入 None”“空容器”，必要时使用专用 sentinel。

最小可运行实验

```
_MISSING = object()

def f(value=_MISSING):
    if value is _MISSING:
        return 'argument not provided'
    if value is None:
        return 'explicit None'
```

易错点 / GOTCHAS

- 把 None 当成 False 的同义词。
- 用 `== None` 作为代码规范。

面试官连续追问

1. 什么时候需要自定义 sentinel?
2. None 与 Optional[T] 的关系是什么?

> **举一反三** 当 None 本身是合法业务值时，专用 sentinel 是区分“缺省”和“显式 None”的常见技巧。

Q07 Python 的 truthy / falsy 规则是什么?

难度：★★ 频率：高 关键词：bool, __bool__, __len__

30-60 秒面试标准回答

布尔上下文并不要求对象本身是 bool。对象可以通过 `__bool__` 定义真假；若没有，通常会尝试 `__len__`；长度为 0 视为 False。

原理拆解

- 常见 falsy 包括 None、False、数值 0、空字符串和空容器。
- 自定义类型可让业务对象具有自然的布尔语义，但过度隐式会降低可读性。
- `if x:` 与 `if x is not None:` 语义不同：前者会把 0、空容器也判为假。

最小可运行实验

```
class Bag:
    def __init__(self, items):
        self.items = items
    def __len__(self):
        return len(self.items)

print(bool(Bag([])))  # False
```

易错点 / GOTCHAS

- 用 `if x:` 判断“是否为 None”，误伤合法的 0/空字符串。
- 让 __bool__ 做昂贵 I/O 或产生副作用。

面试官连续追问

1. 如果 __bool__ 返回非 bool 会怎样?
2. numpy array 为什么不允许模糊的 truth test?

> **举一反三** 面试中要先澄清业务语义：你要判断“存在”“非空”“非零”还是“不是 None”。

Q08 and / or 为什么不一定返回 bool?

难度：★★ 频率：中 关键词：short-circuit, truthiness, expression

30-60 秒面试标准回答

`and`/`or` 是短路表达式，并返回参与运算的某个操作数，而不强制转换成 bool。

原理拆解

- `a or b` 返回第一个 truthy 操作数，否则返回最后一个；`a and b` 返回第一个 falsy 操作数，否则返回最后一个。
- 短路行为常用于默认值与守卫表达式，但要警惕合法 falsy 值。
- `value or default` 会把 0、""、[] 等也替换为 default，因此不等价于“仅当 value is None”。

最小可运行实验

```
timeout = user_timeout or 30  # 若 0 是合法值，这可能是 bug

timeout = 30 if user_timeout is None else user_timeout
```

易错点 / GOTCHAS

- 把 `or` 默认值写法无脑用于所有配置参数。
- 认为表达式结果一定是 True/False。

面试官连续追问

1. 短路求值如何影响有副作用函数?
2. `any`/`all` 与短路有什么关系?

> **举一反三** 简洁表达式只有在业务语义与 falsy 规则完全一致时才是好代码。

Q09 为什么 mutable default argument 是经典陷阱?

难度：★★ 频率：极高 关键词：default args, function object, state

30-60 秒面试标准回答

默认参数表达式在函数定义时求值一次，随后存放在函数对象中复用。因此可变默认值会在多次调用之间共享状态。

原理拆解

- 函数的 `__defaults__` 可以直接观察默认对象，说明它不是“每次调用重新执行表达式”。
- 安全惯例是使用 None 或 sentinel，在函数体内创建新的 list/dict/set。
- 这一机制有时也能被有意用作缓存，但可读性差、线程安全难推理，不推荐作为隐式状态容器。

最小可运行实验

```
def bad(x, items=[]):
    items.append(x)
    return items

print(bad(1))
print(bad(2))  # [1, 2]

def good(x, items=None):
    if items is None:
        items = []
    items.append(x)
    return items
```

易错点 / GOTCHAS

- 认为默认参数在调用时求值。
- 只修复 list，却忘了 dict/set/自定义可变对象同样有问题。

面试官连续追问

1. 默认参数何时求值？装饰器何时求值?
2. dataclass 的 default_factory 为什么与此问题相似?

> **举一反三** 这题经常与 closure late binding 联动考察“定义时 vs 调用时”的时间语义。

Q10 global 和 nonlocal 有什么区别? LEGB 如何工作?

难度：★★ 频率：高 关键词：scope, LEGB, closure

30-60 秒面试标准回答

`global` 把名字声明为模块全局作用域中的绑定；`nonlocal` 指向最近的非全局 enclosing function scope。名字查找通常用 LEGB 作为入门心智模型。

原理拆解

- LEGB = Local, Enclosing, Global, Builtins；但类体、comprehension 等细节需要结合具体语义理解。
- 读取外层变量通常不需要 nonlocal；只有要在内层作用域重新绑定该名字时才需要。
- 滥用 global/nonlocal 会制造隐式共享状态；生产代码更推荐显式对象状态或参数传递。

最小可运行实验

```
x = 1

def outer():
    count = 0
    def inc():
        nonlocal count
        count += 1
        return count
    return inc
```

易错点 / GOTCHAS

- 把 nonlocal 理解为“任意外层变量”。
- 用 global 解决模块之间的状态管理问题。

面试官连续追问

1. 类体中的名字查找和函数作用域有何不同?
2. comprehension 变量为什么不会泄漏到外部作用域?

> **举一反三** 面试答到作用域后，最好补充“显式依赖优于隐式全局状态”的工程观点。

CHAPTER 02

核心容器与哈希表

从 list/tuple/dict/set 的 API 进入复杂度、哈希契约、拷贝语义与工程选择。

本章目标：能够从 API 选择上升到数据结构、复杂度与哈希契约。

本章题目

Q11 list 和 tuple 的区别是什么？ 难度 ★ · 频率 极高 · list, tuple, mutability

Q12 list.append 为什么是摊还 O(1)？ 难度 ★★ · 频率 高 · dynamic array, amortized analysis, complexity

Q13 为什么 list.insert(0, x) / pop(0) 慢？ 难度 ★ · 频率 高 · list, deque, data structure

Q14 Python dict 查找为什么平均 O(1)？ 难度 ★★★ · 频率 极高 · dict, hash table, collision

Q15 什么对象可以作为 dict key？ 难度 ★★ · 频率 极高 · hashable, __hash__, dict key

Q16 为什么实现 __eq__ 后必须认真考虑 __hash__？ 难度 ★★★ · 频率 高 · data model, equality, hash contract

Q17 dict 是否保证插入顺序？ 难度 ★ · 频率 高 · dict, ordering, language spec

Q18 dict.get、setdefault、defaultdict 如何选择？ 难度 ★★ · 频率 高 · dict, defaultdict, API design

Q19 set 为什么能高效查重？ 难度 ★ · 频率 极高 · set, hashing, dedup

Q20 浅拷贝和深拷贝有什么区别？ 难度 ★★ · 频率 极高 · copy, aliasing, deepcopy

Q11 list 和 tuple 的区别是什么?

难度：★ 频率：极高 关键词：list, tuple, mutability

30-60 秒面试标准回答

核心差异是可变性与语义定位：list 适合可变集合；tuple 适合固定结构/记录。tuple 是否可哈希还取决于其元素。

原理拆解

- list 支持原地增删改；tuple 的元素引用序列固定。
- tuple 通常更紧凑，但“更快”不是选择它的首要理由，语义表达更重要。
- tuple 只有在所有元素都可哈希时才可作为 dict key/set 元素。

最小可运行实验

```
point = (10, 20)
queue = [10, 20]

# tuple 内也可能引用可变对象
x = ([1], 2)
```

易错点 / GOTCHAS

- 只回答“list 可变，tuple 不可变”，没有谈 hash/语义。
- 用 tuple 仅仅为了“性能优化”而牺牲可读性。

面试官连续追问

1. namedtuple/dataclass 与 tuple 如何取舍?
2. 为什么 tuple 可以被缓存和共享得更安全?

> **举一反三** 数据结构选择首先表达业务不变量，其次才是微小的空间/速度差异。

Q12 list.append 为什么是摊还 O(1)?

难度：★★ 频率：高 关键词：dynamic array, amortized analysis, complexity

30-60 秒面试标准回答

list 是动态数组模型。大多数 append 直接写入预留容量；偶尔扩容需要重新分配并复制指针，因此单次可能 O(n)，长期序列摊还 O(1)。

原理拆解

- 摊还分析关注一串操作总成本，而不是某一次最坏成本。
- list 存储的是对象引用而不是把所有 Python 对象内容嵌入连续内存。
- 扩容策略是实现细节，面试不必死背具体增长公式，除非题目要求 CPython 源码。

最小可运行实验

```
xs = []
for i in range(1_000_000):
    xs.append(i)
```

易错点 / GOTCHAS

- 说 append 永远 O(1)。
- 把 amortized 和 average-case 混为一谈。

面试官连续追问

1. 为什么 insert(0, x) 不是 O(1)?
2. deque 为什么适合两端操作?

> **举一反三** 复杂度回答要注明“摊还/平均/最坏”哪一种口径。

Q13 为什么 list.insert(0, x) / pop(0) 慢?

难度：★ 频率：高 关键词：list, deque, data structure

30-60 秒面试标准回答

list 的元素引用位于连续数组中，头部插入/删除需要移动大量引用，复杂度 O(n)。两端频繁操作应考虑 deque。

原理拆解

- 尾部 append/pop 通常是摊还 O(1)。
- deque 是分块双端队列，适合 appendleft/popleft，但不擅长任意位置随机访问。
- 数据结构选择应从操作模式出发，而不是“哪个容器更快”的笼统比较。

最小可运行实验

```
from collections import deque
q = deque()
q.append(1)
q.appendleft(0)
q.popleft()
```

易错点 / GOTCHAS

- 用 list 作为高频 FIFO 队列并不断 pop(0)。
- 把 deque 当成 list 的完全替代品。

面试官连续追问

1. 实现 BFS 用什么容器?
2. 随机访问与两端操作为什么存在权衡?

> **举一反三** 面试官常借这题观察你能否从 API 跳到数据结构和复杂度。

Q14 Python dict 查找为什么平均 O(1)?

难度：★★★ 频率：极高 关键词：dict, hash table, collision

30-60 秒面试标准回答

dict 使用哈希表：先计算 key 的 hash，再通过探测定位槽位，并在冲突时继续比较。平均查找/插入 O(1)，最坏情况理论上可退化。

原理拆解

- hash 先缩小候选位置，最终 equality comparison 仍参与确认 key。
- 冲突不可避免，哈希表依赖容量、探测策略和 resize 维持较低平均成本。
- 自定义 key 的 `__hash__`/`__eq__` 如果很慢，会直接拖累 dict 操作；错误契约会导致逻辑问题。

最小可运行实验

```
d = {'user_id': 42}
print(d['user_id'])

class Key:
    def __hash__(self): ...
    def __eq__(self, other): ...
```

易错点 / GOTCHAS

- 只背“dict 是 O(1)”而不注明平均意义。
- 认为 hash 相同就意味着对象相等。

面试官连续追问

1. hash collision 怎么处理?
2. 为什么 dict 需要扩容?
3. Python 的 hash randomization 解决什么问题?

> **举一反三** 性能诊断中，要考虑 key 的哈希/比较成本，而不只是容器本身。

Q15 什么对象可以作为 dict key?

难度：★★ 频率：极高 关键词：hashable, __hash__, dict key

30-60 秒面试标准回答

dict key 必须 hashable。核心要求是哈希值在参与哈希表期间稳定，且相等对象必须有相同哈希值。

原理拆解

- str/int/bytes/frozenset 等通常可哈希；list/dict/set 因可变而不可哈希。
- tuple 是否可哈希取决于元素；包含 list 的 tuple 不可哈希。
- 自定义 mutable 对象即使技术上实现 __hash__ 也要非常谨慎，尤其不能让参与 equality/hash 的字段在入表后变化。

最小可运行实验

```
print(hash((1, 2)))
# hash([1], 2))  # TypeError
```

易错点 / GOTCHAS

- 把“immutable”当作 hashable 的唯一形式定义。
- 实现 __hash__ 后允许相关字段随意变化。

面试官连续追问

1. `__eq__` 与 `__hash__` 的契约是什么?
2. frozenset 为什么适合做集合型 key?

> **举一反三** 哈希键设计是缓存、去重、memoization 与 ORM identity 的共同基础。

Q16 为什么实现 __eq__ 后必须认真考虑 __hash__?

难度：★★★ 频率：高 关键词：data model, equality, hash contract

30-60 秒面试标准回答

哈希容器依赖契约：若 a == b，则 hash(a) 必须等于 hash(b)。违反契约会让“逻辑上相等”的对象落到不同查找路径。

原理拆解

- Python 会对某些重写 __eq__ 的类默认禁用可哈希性，防止无意破坏契约。
- 哈希值不要求不同对象都不同；碰撞是允许的。
- 用于 hash/equality 的字段最好是稳定不变的值语义字段。

最小可运行实验

```
class UserKey:
    def __init__(self, uid): self.uid = uid
    def __eq__(self, other):
        return isinstance(other, UserKey) and self.uid == other.uid
    def __hash__(self):
        return hash(self.uid)
```

易错点 / GOTCHAS

- 误以为不同对象必须有不同 hash。
- 把可变列表内容纳入 hash，入 set 后再修改。

面试官连续追问

1. dataclass(frozen=True) 如何处理 hash?
2. identity hash 与 value hash 有什么区别?

> **举一反三** 面试中给出契约，再说明“稳定字段 + 等价语义”比死背实现细节更重要。

Q17 dict 是否保证插入顺序?

难度：★ 频率：高 关键词：dict, ordering, language spec

30-60 秒面试标准回答

现代 Python 的 dict 保持插入顺序；从 Python 3.7 起这是语言层面的保证。保持插入顺序不等于按 key 排序。

原理拆解

- 更新已有 key 的 value 通常不改变其原位置；删除再重新插入会形成新的插入位置。
- 如果需要按 key/value 排序，仍应显式使用 sorted。
- 依赖顺序时应让代码意图清晰，而不是把实现偶然行为当作业务协议。

最小可运行实验

```
d = {}
d['b'] = 2
d['a'] = 1
print(list(d))  # ['b', 'a']
print(sorted(d)) # ['a', 'b']
```

易错点 / GOTCHAS

- 说“dict 是无序的”作为现代 Python 标准答案。
- 把 insertion order 与 sorted order 混淆。

面试官连续追问

1. OrderedDict 现在还有哪些特殊价值?
2. 删除再插入会发生什么?

> **举一反三** 版本题要给出版本边界，而不是只说“以前怎样”。

Q18 dict.get、setdefault、defaultdict 如何选择?

难度：★★ 频率：高 关键词：dict, defaultdict, API design

30-60 秒面试标准回答

`get` 适合只读默认值；`setdefault` 适合一次性“取或创建”；`defaultdict` 适合长期、重复的自动初始化模式。

原理拆解

- `get` 不会把默认值写入字典。
- `setdefault` 会在缺失时插入默认对象，复杂嵌套时可读性迅速下降。
- defaultdict 的 default_factory 会在 `__getitem__` 缺失时创建值，但 `get` 行为需要单独理解。

最小可运行实验

```
from collections import defaultdict
index = defaultdict(list)
for word, doc_id in pairs:
    index[word].append(doc_id)
```

易错点 / GOTCHAS

- 为了少写一行代码滥用 setdefault 形成难读表达式。
- 不知道 get 和 defaultdict 的缺失行为并不完全一样。

面试官连续追问

1. Counter 什么时候比 defaultdict(int) 更合适?
2. 如何避免 default_factory 隐式修改状态?

> **举一反三** 容器 API 选择应强调“是否希望读取动作产生写入副作用”。

Q19 set 为什么能高效查重?

难度：★ 频率：极高 关键词：set, hashing, dedup

30-60 秒面试标准回答

set 使用哈希表实现成员检测，平均 `x in s` 和 `s.add(x)` 都接近 O(1)，所以常把二重循环查重优化为线性扫描。

原理拆解

- set 牺牲了位置/重复信息来换取高效成员查询。
- 元素必须 hashable。
- 当数据极大、允许概率误差时，Bloom Filter 等结构可能进一步降低内存，但那已超出内置 set 的语义。

最小可运行实验

```
def has_duplicate(xs):
    seen = set()
    for x in xs:
        if x in seen:
            return True
        seen.add(x)
    return False
```

易错点 / GOTCHAS

- 忽略元素不可哈希问题。
- 对极大对象集合只考虑算法复杂度，不考虑内存。

面试官连续追问

1. 如何保序去重?
2. set 与 frozenset 的区别?

> **举一反三** 算法题里从 O(n^2) 到 O(n) 的常见突破口就是“用空间换时间”的哈希表。

Q20 浅拷贝和深拷贝有什么区别?

难度：★★ 频率：极高 关键词：copy, aliasing, deepcopy

30-60 秒面试标准回答

浅拷贝创建新的外层容器，但通常复用内部对象引用；深拷贝会递归复制可复制的对象图，并维护 memo 处理共享引用/循环。

原理拆解

- `list(x)`、切片、`copy.copy` 对 list 都属于浅层语义。
- deepcopy 不代表“适合所有对象”：文件、socket、锁、数据库连接等资源不能靠深拷贝解决。
- 深拷贝可能昂贵，也可能破坏有意共享的对象关系。

最小可运行实验

```
import copy
a = [[1], [2]]
b = copy.copy(a)
c = copy.deepcopy(a)
a[0].append(9)
print(b)  # [[1, 9], [2]]
print(c)  # [[1], [2]]
```

易错点 / GOTCHAS

- 看到嵌套结构就无脑 deepcopy。
- 误以为浅拷贝之后完全独立。

面试官连续追问

1. deepcopy 如何处理循环引用?
2. 不可变对象为何常被复用?

> **举一反三** 工程中更推荐清晰的 ownership/immutable design，而不是靠 deepcopy 修复不清晰的共享关系。

CHAPTER 03

函数、闭包与装饰器

把函数当一等对象理解，掌握闭包、late binding、装饰器、缓存与调用签名。

本章目标：能够自己写正确的 decorator/closure，并解释定义时与调用时语义。

本章题目

Q21 *args 和 **kwargs 的本质是什么？ 难度 ★ · 频率 极高 · call syntax, packing, unpacking

Q22 positional-only 与 keyword-only 参数解决什么问题？ 难度 ★★ · 频率 中 · signature, API design, PEP570

Q23 为什么说函数是一等对象？ 难度 ★ · 频率 高 · first-class function, callable, higher-order

Q24 什么是闭包？ 难度 ★★ · 频率 极高 · closure, lexical scope, state

Q25 late binding closure 为什么会得到 [2,2,2]？ 难度 ★★★ · 频率 极高 · closure, late binding, loop

Q26 装饰器本质是什么？ 难度 ★★ · 频率 极高 · decorator, higher-order function, binding

Q27 为什么 decorator 通常需要 functools.wraps？ 难度 ★★ · 频率 高 · wraps, metadata, introspection

Q28 如何实现带参数装饰器？ 难度 ★★ · 频率 高 · decorator factory, closure, configuration

Q29 lambda 与普通 def 的区别是什么？ 难度 ★ · 频率 中 · lambda, expression, readability

Q30 functools.lru_cache 有哪些工程陷阱？ 难度 ★★★ · 频率 高 · cache, hashable, lifetime

Q21 *args 和 **kwargs 的本质是什么?

难度：★ 频率：极高 关键词：call syntax, packing, unpacking

30-60 秒面试标准回答

`*args` 收集额外位置参数为 tuple，`**kwargs` 收集额外关键字参数为 dict；`*`/`**` 同时也是调用与字面量中的 unpacking 语法。

原理拆解

- 名字 args/kwargs 只是惯例，真正有语义的是 `*` 和 `**`。
- 过度使用 `**kwargs` 会让 API 契约模糊，类型检查和 IDE 补全变差。
- 包装器/装饰器中常需要完整转发参数，因此它们尤其常见。

最小可运行实验

```
def f(a, *args, **kwargs):
    print(a, args, kwargs)

params = {'debug': True}
f(1, 2, 3, **params)
```

易错点 / GOTCHAS

- 把 args/kwargs 当关键字。
- 所有公共 API 都用 **kwargs 逃避明确签名。

面试官连续追问

1. 函数调用时 `*seq` 与 `**mapping` 的校验规则?
2. 如何给 decorator 精确保留参数类型?

> **举一反三** 公共 API 应尽量显式；转发层才更适合使用 args/kwargs。

Q22 positional-only 与 keyword-only 参数解决什么问题?

难度：★★ 频率：中 关键词：signature, API design, PEP570

30-60 秒面试标准回答

`/` 左侧参数只能按位置传；`*` 右侧参数只能按关键字传。它们让 API 的调用契约更精确，也给未来参数重命名/扩展留下空间。

原理拆解

- positional-only 可避免调用者依赖内部参数名。
- keyword-only 适合布尔开关、单位、超时等需要显式说明含义的参数。
- 设计函数签名时，可读性与向后兼容性比“参数越灵活越好”重要。

最小可运行实验

```
def connect(host, /, port=443, *, timeout=5, verify=True):
    ...

connect('example.com', timeout=2)
```

易错点 / GOTCHAS

- 看到 `/` 误以为是除法。
- 多个 bool 参数仍允许位置传，导致 `f(x, True, False)` 难读。

面试官连续追问

1. Python 内置函数为什么常用 positional-only?
2. 如何检查函数 Signature?

> **举一反三** 成熟 API 用签名表达意图，减少靠文档猜参数含义。

Q23 为什么说函数是一等对象？

难度：★ 频率：高 关键词：first-class function, callable, higher-order

30-60 秒面试标准回答

函数可以赋给变量、放入容器、作为参数和返回值，因此可以构建高阶函数、回调、策略表和装饰器。

原理拆解

- 函数对象自身携带 `__name__`、`__doc__`、annotations、defaults、closure 等元数据。
- 一等函数让行为可以像数据一样组合，但过度动态注册也会增加追踪难度。
- `callable(obj)` 的范围比函数更广，实现 `__call__` 的对象同样可调用。

最小可运行实验

```
def add1(x): return x + 1
ops = {'inc': add1}
print(ops['inc'](3))

class Multiplier:
    def __init__(self, k): self.k = k
    def __call__(self, x): return self.k * x
```

易错点 / GOTCHAS

- 把 callable 等同于 function。
- 回调层级过深导致控制流难追踪。

面试官连续追问

1. 函数对象有哪些重要属性？
2. functor 与 closure 有何取舍？

> **举一反三** 策略模式在 Python 中常可直接用函数表实现，不一定需要复杂类层次。

Q24 什么是闭包?

难度：★★ 频率：极高 关键词：closure, lexical scope, state

30-60 秒面试标准回答

闭包是函数与其捕获的词法环境的组合，使内层函数在外层调用结束后仍能访问被捕获的自由变量。

原理拆解

- 闭包捕获的是变量绑定关系，不是简单“把值复制进去”。
- `__closure__` 与 code object 的 freevars 能帮助理解实现。
- 闭包适合轻量状态与函数工厂；状态复杂时类通常更可维护。

最小可运行实验

```
def make_multiplier(k):
    def mul(x):
        return k * x
    return mul

times3 = make_multiplier(3)
print(times3(10))
```

易错点 / GOTCHAS

- 说闭包就是“嵌套函数”。
- 忽略自由变量是否真正被捕获。

面试官连续追问

1. 闭包变量何时释放?
2. nonlocal 如何修改闭包状态?

> **举一反三** 装饰器往往就是“高阶函数 + 闭包”的组合。

Q25 late binding closure 为什么会得到 [2,2,2]?

难度：★★★ 频率：极高 关键词：closure, late binding, loop

30-60 秒面试标准回答

闭包通常在调用时读取自由变量当前绑定，而不是在创建 lambda 时冻结循环变量的当时值。因此循环结束后多个函数看到同一个最终 i。

原理拆解

- 问题的本质是绑定时间，不是 lambda 特有；普通内层 def 同样会发生。
- 使用默认参数 `i=i` 可利用默认值定义时求值的特性冻结当轮对象。
- 也可用函数工厂为每轮创建新的作用域。

最小可运行实验

```
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])  # [2, 2, 2]

funcs = [lambda i=i: i for i in range(3)]
print([f() for f in funcs])  # [0, 1, 2]
```

易错点 / GOTCHAS

- 误以为 lambda 会自动捕获“当前值”。
- 修复时引入同一个可变默认对象导致新问题。

面试官连续追问

1. 为什么默认参数恰好能修?
2. async loop 中创建回调如何避免类似问题?

> **举一反三** 这题与 mutable default argument 一起考，能检验候选人是否理解“定义时/调用时”。

Q26 装饰器本质是什么?

难度：★★ 频率：极高 关键词：decorator, higher-order function, binding

30-60 秒面试标准回答

`@decorator` 是函数/类定义完成后的转换语法。对函数而言，`@d` 近似等价于 `f = d(f)`。

原理拆解

- 装饰器可以返回任意 callable，不要求返回原函数。
- 多个装饰器的应用顺序与嵌套函数调用类似，需要能够推导。
- 生产装饰器应考虑 metadata、异常语义、同步/异步函数差异与类型签名。

最小可运行实验

```
def trace(fn):
    def wrapper(*args, **kwargs):
        print('call', fn.__name__)
        return fn(*args, **kwargs)
    return wrapper

@trace
def add(a, b): return a + b
```

易错点 / GOTCHAS

- 只会写模板，解释不出 `f = d(f)`。
- 装饰 async function 却返回普通同步 wrapper，破坏语义。

面试官连续追问

1. 两个装饰器的执行顺序?
2. 类装饰器与函数装饰器有何共同点?

> **举一反三** 先说语法等价关系，再谈工程问题，是面试中最清晰的回答结构。

Q27 为什么 decorator 通常需要 functools.wraps?

难度：★★ 频率：高 关键词：wraps, metadata, introspection

30-60 秒面试标准回答

`functools.wraps` 让 wrapper 保留被包装函数的重要元数据，并设置 `__wrapped__`，便于 introspection、调试、文档和某些框架正确工作。

原理拆解

- 不使用 wraps 时 `__name__`、`__doc__` 等会变成 wrapper 的信息。
- `inspect.signature` 等工具可以借 `__wrapped__` 追踪原函数。
- wraps 不会自动解决参数类型保留问题；静态类型层面可进一步使用 ParamSpec。

最小可运行实验

```
from functools import wraps

def deco(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        return fn(*args, **kwargs)
    return wrapper
```

易错点 / GOTCHAS

- 把 wraps 当成“性能优化”。
- 用了 wraps 就认为类型签名也完全安全。

面试官连续追问

1. `__wrapped__` 有什么用?
2. 如何给 decorator 写 ParamSpec 类型?

> **举一反三** 反射型框架非常依赖函数元数据，装饰器不能只关注运行结果。

Q28 如何实现带参数装饰器?

难度：★★ 频率：高 关键词：decorator factory, closure, configuration

30-60 秒面试标准回答

带参数装饰器通常是“装饰器工厂”：第一层接收配置，返回真正接收函数的 decorator，再返回 wrapper。

原理拆解

- `@retry(3)` 先计算 `retry(3)`，得到 decorator，再用它转换函数。
- 配置参数本身可能是可变对象，需要同样考虑生命周期。
- 生产 retry 还要考虑异常类型、退避、幂等性、最大总时长和可观测性。

最小可运行实验

```
from functools import wraps

def retry(times):
    def deco(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            last = None
            for _ in range(times):
                try:
                    return fn(*args, **kwargs)
                except Exception as e:
                    last = e
            raise last
        return wrapper
    return deco
```

易错点 / GOTCHAS

- 任何异常都无脑 retry。
- 对非幂等写操作重试导致重复副作用。

面试官连续追问

1. 如何支持 async retry?
2. 指数退避和 jitter 为什么重要?

> **举一反三** 真正高级的回答会把语言机制连接到分布式系统中的幂等与重试语义。

Q29 lambda 与普通 def 的区别是什么?

难度：★ 频率：中 关键词：lambda, expression, readability

30-60 秒面试标准回答

lambda 创建匿名函数，函数体限制为单个表达式；它不是更快的函数语法。简单 key/callback 很适合，复杂逻辑应使用 def。

原理拆解

- lambda 仍然是 function object，也受 closure/late binding 规则影响。
- 匿名性会降低 trace/profile 中的可读性。
- 把复杂控制流塞进表达式往往是 code smell。

最小可运行实验

```
users = [{'name': 'a', 'score': 9}, {'name': 'b', 'score': 7}]
users.sort(key=lambda u: u['score'], reverse=True)
```

易错点 / GOTCHAS

- 认为 lambda 性能更高。
- 在 lambda 中构造复杂嵌套表达式以追求“一行代码”。

面试官连续追问

1. lambda 是否能有默认参数?
2. 为什么 Python lambda 只允许 expression?

> **举一反三** 面试代码首先追求正确、清晰、可解释，不追求“最短”。

Q30 functools.lru_cache 有哪些工程陷阱?

难度：★★★ 频率：高 关键词：cache, hashable, lifetime

30-60 秒面试标准回答

`lru_cache` 通过参数作为 key 缓存返回值，适用于同输入同输出、参数可哈希且结果可安全复用的函数。它会持有 key/value，因此也影响对象生命周期和内存。

原理拆解

- 有外部可变状态、时间、权限或 I/O 依赖的函数并非天然可缓存。
- 装饰实例方法时 self 是 key 的一部分，缓存可间接持有实例。
- 缓存需要明确失效策略；无限缓存不是“免费加速”。

最小可运行实验

```
from functools import lru_cache

@lru_cache(maxsize=1024)
def parse_schema(text: str):
    return expensive_parse(text)

# parse_schema.cache_info()
# parse_schema.cache_clear()
```

易错点 / GOTCHAS

- 给会返回可变对象的函数加 cache 后让调用者修改结果。
- 忽略缓存失效和内存上限。

面试官连续追问

1. TTL cache 如何实现?
2. cache stampede 如何处理?
3. cached_property 与 lru_cache 差异?

> **举一反三** 缓存题不要只谈命中率，还要谈一致性、失效、容量和生命周期。

CHAPTER 04

OOP 与 Python Data Model

理解属性访问、descriptor、MRO、super、metaclass 与 __slots__ 等高级机制。

本章目标：能够推导属性访问、method binding、MRO 与元编程机制。

本章题目

Q31 __new__ 和 __init__ 有什么区别? 难度 ★★★ · 频率 高 · object creation, __new__, __init__

Q32 实例方法、classmethod、staticmethod 如何选择? 难度 ★★ · 频率 极高 · method binding, classmethod, staticmethod

Q33 Python 的“private”是怎么实现的? 难度 ★★ · 频率 高 · name mangling, encapsulation, convention

Q34 __getattribute__ 和 __getattr__ 有什么区别? 难度 ★★★★ · 频率 高 · attribute lookup, __getattribute__, __getattr__

Q35 什么是 descriptor? 为什么它如此重要? 难度 ★★★★★ · 频率 高 · descriptor, __get__, property

Q36 property 是如何工作的? 难度 ★★★ · 频率 高 · property, descriptor, encapsulation

Q37 MRO 是什么? C3 linearization 解决什么问题? 难度 ★★★★ · 频率 高 · MRO, multiple inheritance, C3

Q38 super() 为什么不是“调用父类”? 难度 ★★★★ · 频率 高 · super, MRO, cooperative inheritance

Q39 什么是 metaclass? 什么时候真的需要? 难度 ★★★★★ · 频率 中 · metaclass, type, class creation

Q40 __slots__ 有什么用? 有哪些坑? 难度 ★★★★ · 频率 高 · __slots__, memory, descriptor

Q31 __new__ 和 __init__ 有什么区别?

难度：★★★ 频率：高 关键词：object creation, __new__, __init__

30-60 秒面试标准回答

`__new__` 负责创建/返回实例；`__init__` 接收已经创建的实例并初始化它。不可变类型的定制通常要在 __new__ 阶段决定最终值。

原理拆解

- `__init__` 不是构造函数意义上的“分配对象”步骤，也不应返回其它值。
- `__new__` 是静态式创建入口，通常调用 `super().__new__(cls)`。
- singleton/flyweight 可以借 __new__ 控制实例，但要谨慎处理初始化多次调用。

最小可运行实验

```
class PositiveInt(int):
    def __new__(cls, value):
        value = max(0, int(value))
        return super().__new__(cls, value)
```

易错点 / GOTCHAS

- 把 __init__ 说成真正负责创建对象。
- __new__ 返回非 cls 实例时仍假定 __init__ 一定执行。

面试官连续追问

1. 元类的 `__call__` 与 __new__/__init__ 如何串联?
2. immutable subclass 为什么依赖 __new__?

> **举一反三** 回答时画出 Class(...) -> __new__ -> __init__ 的链路最清晰。

Q32 实例方法、classmethod、staticmethod 如何选择?

难度：★★ 频率：极高 关键词：method binding, classmethod, staticmethod

30-60 秒面试标准回答

实例方法绑定 self；classmethod 绑定 cls，常用于 alternate constructor；staticmethod 不绑定实例/类，适合作为与类概念相关的纯工具函数。

原理拆解

- classmethod 通过 cls 构造对象，天然支持子类多态，比硬编码类名更好。
- staticmethod 如果与类没有概念关系，放模块级函数往往更简单。
- 三者都可以看作 descriptor 行为的不同封装。

最小可运行实验

```
class User:
    def __init__(self, name): self.name = name

    @classmethod
    def from_email(cls, email):
        return cls(email.split('@')[0])

    @staticmethod
    def valid_name(name):
        return bool(name.strip())
```

易错点 / GOTCHAS

- alternate constructor 内硬编码 `User(...)` 而不是 cls。
- 所有 utility 都塞进 class 变成 Java 风格命名空间。

面试官连续追问

1. classmethod 在继承时为什么更有优势?
2. 绑定方法对象是什么?

> **举一反三** 选择标准是“这个行为依赖实例状态、类状态，还是都不依赖”。

Q33 Python 的“private”是怎么实现的?

难度：★★ 频率：高 关键词：name mangling, encapsulation, convention

30-60 秒面试标准回答

单下划线 `_x` 是非公开约定；双下划线 `__x` 触发 name mangling，主要防止子类命名冲突，不是安全边界。

原理拆解

- `__x` 通常会变为 `_ClassName__x` 形式，因此仍可访问。
- Python 封装更依赖 API 约定、property、模块边界与文档，而不是强制访问控制。
- 名称改写对继承层次中的意外覆盖有价值。

最小可运行实验

```
class A:
    def __init__(self):
        self._internal = 1
        self.__mangled = 2

a = A()
print(a._A__mangled)
```

易错点 / GOTCHAS

- 把双下划线称为“真正 private”。
- 把单下划线理解为解释器禁止访问。

面试官连续追问

1. 模块级 `_name` 有什么意义?
2. 双下划线方法 `__len__` 与 name mangling 是一回事吗?

> **举一反三** Python 的封装哲学更偏“明确约定 + 工具支持”，不是语言层面的绝对隔离。

Q34 __getattribute__ 和 __getattr__ 有什么区别?

难度：★★★★ 频率：高 关键词：attribute lookup, __getattribute__, __getattr__

30-60 秒面试标准回答

`__getattribute__` 参与几乎所有实例属性访问；`__getattr__` 只在正常查找失败时作为 fallback 调用。

原理拆解

- 在 __getattribute__ 内再次写 `self.x` 很容易无限递归，应委托 `object.__getattribute__`。
- __getattr__ 适合 lazy/proxy/default attribute，但会让 typo 更难暴露，必须谨慎。
- 理解 attribute lookup 后才能真正理解 descriptor、property、method binding。

最小可运行实验

```
class Proxy:
    def __init__(self, target):
        object.__setattr__(self, '_target', target)

    def __getattr__(self, name):
        return getattr(self._target, name)
```

易错点 / GOTCHAS

- 在 __getattribute__ 中直接 `return self.__dict__[name]` 导致递归/漏掉 descriptor。
- __getattr__ 对所有不存在属性静默返回 None，隐藏 bug。

面试官连续追问

1. 数据描述符与实例 __dict__ 谁优先?
2. `__setattr__` 的递归陷阱是什么?

> **举一反三** 高级 Python 面试常用这题判断候选人是否理解“属性访问是一套协议”。

Q35 什么是 descriptor？为什么它如此重要？

难度：★★★★★ 频率：高 关键词：descriptor, __get__, property

30-60 秒面试标准回答

实现 `__get__`、`__set__`、`__delete__` 中相应方法的对象可参与 descriptor protocol。property、函数方法绑定、classmethod、staticmethod、slots、许多 ORM field 都建立在该机制上。

原理拆解

- data descriptor（定义 __set__/__delete__）通常优先于实例 __dict__；non-data descriptor 通常在实例属性之后。
- 函数对象在类中作为 non-data descriptor，使 `obj.method` 产生绑定方法。
- descriptor 将“属性语法”与“存储/验证/计算策略”解耦，是 Python Data Model 的核心。

最小可运行实验

```
class Positive:
    def __set_name__(self, owner, name):
        self.name = '_' + name
    def __get__(self, obj, owner=None):
        if obj is None: return self
        return getattr(obj, self.name)
    def __set__(self, obj, value):
        if value < 0: raise ValueError('must be >= 0')
        setattr(obj, self.name, value)
```

易错点 / GOTCHAS

- 把 descriptor 等同于 decorator。
- 解释不出为什么普通函数放进 class 后会自动绑定 self。

面试官连续追问

1. data vs non-data descriptor 的优先级？
2. `__set_name__` 何时调用？
3. ORM Field 如何利用 descriptor？

> **举一反三** 能解释 method binding + property + ORM 三个例子，基本就真正理解 descriptor 了。

Q36 property 是如何工作的?

难度：★★★ 频率：高 关键词：property, descriptor, encapsulation

30-60 秒面试标准回答

property 是 descriptor 的标准封装，让调用方仍使用 `obj.attr` 语法，而类内部可以插入计算、验证、兼容逻辑。

原理拆解

- property 适合从公开数据属性平滑演进为计算属性，避免调用 API 从 `x.age` 改成 `x.get_age()`。
- setter 中应保持不变量，但不要隐藏昂贵网络调用等令人意外的副作用。
- 只读 property 并不意味着底层对象不可变。

最小可运行实验

```
class Temperature:
    def __init__(self, c): self._c = c
    @property
    def celsius(self): return self._c
    @celsius.setter
    def celsius(self, value):
        if value < -273.15: raise ValueError
        self._c = value
```

易错点 / GOTCHAS

- 把昂贵 I/O 放在普通 property 中，让一次属性访问变得不可预测。
- setter 只做赋值不维持业务不变量。

面试官连续追问

1. cached_property 与 property 区别?
2. property 与 descriptor class 如何互换实现?

> **举一反三** API 设计目标是保持调用方语义稳定，同时允许实现演进。

Q37 MRO 是什么？C3 linearization 解决什么问题？

难度：★★★★ 频率：高 关键词：MRO, multiple inheritance, C3

30-60 秒面试标准回答

MRO 决定多继承中的属性/方法查找顺序。Python 使用 C3 linearization 保证局部优先顺序、一致性与单调性。

原理拆解

- 可通过 `Class.__mro__` 或 `Class.mro()` 查看线性化结果。
- 多继承中“父类”不是一棵简单树，因此不能只凭直觉猜 super 下一站。
- 不一致的继承关系会在类创建时失败，而不是产生随机顺序。

最小可运行实验

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass
print(D.__mro__)
```

易错点 / GOTCHAS

- 认为 MRO 就是简单“从左到右 DFS”。
- 在复杂继承中硬编码父类调用。

面试官连续追问

1. diamond inheritance 中为什么不会重复调用同一 cooperative method?
2. C3 的单调性有什么价值?

> **举一反三** 面试里不用手推复杂 C3 算法，但必须知道它决定 super/属性查找。

Q38 super() 为什么不是“调用父类”？

难度：★★★★ 频率：高 关键词：super, MRO, cooperative inheritance

30-60 秒面试标准回答

`super()` 的核心是“从当前类在 MRO 中的位置继续查找”，并不等于固定调用语法上的直接父类。

原理拆解

- cooperative multiple inheritance 要求各层使用兼容签名并继续 `super()` 链。
- 硬编码 `Base.method(self)` 会绕开 MRO，使 mixin/diamond 组合失效。
- 无参 super 依赖编译器提供的 `__class__` cell，嵌套场景需理解上下文。

最小可运行实验

```
class A:
    def f(self): print('A')
class B(A):
    def f(self):
        print('B')
        super().f()
```

易错点 / GOTCHAS

- 把 super 翻译成“父类对象”。
- 一半类用 super、一半硬编码父类导致调用链断裂。

面试官连续追问

1. mixin 应如何设计签名？
2. super 在 classmethod 中如何表现？

> **举一反三** 高级回答要从“直接父类”升级为“协作式 MRO 链”。

Q39 什么是 metaclass？什么时候真的需要？

难度：★★★★★ 频率：中 关键词：metaclass, type, class creation

30-60 秒面试标准回答

metaclass 是创建 class object 的类；默认大多数类的 metaclass 是 `type`。它可拦截类定义过程，用于注册、验证、生成方法等框架级能力。

原理拆解

- `type(MyClass) is type` 展示了“类也是对象”。
- 类装饰器、`__init_subclass__`、descriptor 往往能解决大部分需求，metaclass 应作为更底层工具。
- 多 metaclass 组合会出现 metaclass conflict，因此框架设计要谨慎。

最小可运行实验

```
class Registry(type):
    classes = {}
    def __new__(mcls, name, bases, ns):
        cls = super().__new__(mcls, name, bases, ns)
        mcls.classes[name] = cls
        return cls
```

易错点 / GOTCHAS

- 为了“高级”而用 metaclass。
- 混淆 instance -> class -> metaclass 三层关系。

面试官连续追问

1. `__init_subclass__` 与 metaclass 如何取舍?
2. ABCMeta 为什么是 metaclass?

> **举一反三** 最好的 metaclass 面试答案通常包含一句：“能不用就不用，优先更简单机制”。

Q40 __slots__ 有什么用？有哪些坑？

难度：★★★★ 频率：高 关键词：__slots__, memory, descriptor

30-60 秒面试标准回答

`__slots__` 声明固定实例属性槽位，可避免每个实例自动创建普通 `__dict__`（除非显式包含或继承而来），常用于大量轻量实例降低内存。

原理拆解

- slot 本身通过 descriptor 参与属性访问。
- 继承自没有 slots 的基类时，子类实例通常仍会有 __dict__；多继承组合也更复杂。
- slots 不是简单“性能开关”，会影响动态属性、弱引用、序列化/框架兼容。

最小可运行实验

```
class Point:
    __slots__ = ('x', 'y')
    def __init__(self, x, y):
        self.x, self.y = x, y
```

易错点 / GOTCHAS

- 声称 slots 在任何场景都更快。
- 忘记需要弱引用时可能要考虑 `__weakref__`。

面试官连续追问

1. slots 与 dataclass(slots=True) 有何关系?
2. 为什么继承设计会影响 slots 效果?

> **举一反三** 是否使用 slots 应通过对象数量、内存 profile 和框架兼容性共同决定。

CHAPTER 05

迭代器、生成器与资源管理

掌握惰性计算、迭代协议、generator delegation 与 context manager。

本章目标：能够构建惰性数据流与异常安全的资源生命周期。

本章题目

Q41 iterable 和 iterator 有什么区别？ 难度 ★★ · 频率 极高 · iterable, iterator, protocol

Q42 iter(x) 实际发生了什么？ 难度 ★★★ · 频率 高 · iteration protocol, __iter__, __getitem__

Q43 generator 与普通函数有什么区别？ 难度 ★★ · 频率 极高 · generator, yield, suspension

Q44 为什么 generator 通常更省内存？ 难度 ★★ · 频率 极高 · lazy evaluation, streaming, memory

Q45 generator.send() 有什么语义？ 难度 ★★★★ · 频率 中 · send, generator protocol, coroutine history

Q46 yield from 不只是语法糖吗？ 难度 ★★★★ · 频率 中 · yield from, delegation, generator

Q47 为什么 generator/iterator 通常只能消费一次？ 难度 ★★ · 频率 高 · stateful iterator, exhaustion, API

Q48 context manager 的核心协议是什么？ 难度 ★★ · 频率 极高 · context manager, __enter__, __exit__

Q49 with 为什么比手写 try/finally 更好？ 难度 ★★ · 频率 高 · RAII-like, resource management, API

Q50 contextlib.contextmanager 是如何把 generator 变成上下文管理器的？ 难度 ★★★ · 频率 中 · contextlib, generator, cleanup

Q41 iterable 和 iterator 有什么区别?

难度：★★ 频率：极高 关键词：iterable, iterator, protocol

30-60 秒面试标准回答

iterable 能产生 iterator；iterator 维护遍历状态并实现 `__next__`。list 是 iterable，但通常不是 iterator。

原理拆解

- `iter(obj)` 获取 iterator；`next(it)` 推进并在结束时引发 StopIteration。
- iterator 的 `__iter__` 通常返回自身，因此 iterator 本身也是 iterable。
- 可重复遍历的 container 与一次性消耗的 iterator 在 API 设计上差别很大。

最小可运行实验

```
xs = [10, 20]
it = iter(xs)
print(next(it))
print(next(it))
# next(it) -> StopIteration
```

易错点 / GOTCHAS

- 把 list 说成 iterator。
- API 接受 iterator 后偷偷消费一次，调用者再用时发现为空。

面试官连续追问

1. 如何实现自定义 iterator?
2. 迭代器为何适合流式数据?

> **举一反三** 函数参数如果会消费 iterator，最好在命名/文档中明确这一事实。

Q42 iter(x) 实际发生了什么?

难度：★★★ 频率：高 关键词：iteration protocol, __iter__, __getitem__

30-60 秒面试标准回答

通常先寻找对象的 `__iter__` 获得 iterator；Python 还保留对某些 sequence protocol 的兼容路径。

原理拆解

- iterator 应在 `__next__` 中产生值或抛 StopIteration。
- for 循环会自动调用 iter/next 并捕获结束信号。
- 自定义容器最好返回独立 iterator，而不是把容器本身也做成一次性 iterator，除非语义就是流。

最小可运行实验

```
class CountDown:
    def __init__(self, n): self.n = n
    def __iter__(self): return self
    def __next__(self):
        if self.n <= 0: raise StopIteration
        self.n -= 1
        return self.n + 1
```

易错点 / GOTCHAS

- 在 __next__ 结束时返回 None 而不是 raise StopIteration。
- 容器和 iterator 状态混在一起导致不能嵌套遍历。

面试官连续追问

1. for 语句如何展开?
2. 为什么 `iter(it) is it` 是常见约定?

> **举一反三** 协议题要能从语法糖还原到底层方法调用。

Q43 generator 与普通函数有什么区别?

难度：★★ 频率：极高 关键词：generator, yield, suspension

30-60 秒面试标准回答

含 `yield` 的函数调用会返回 generator object；函数体按迭代推进执行，可在 yield 点暂停并保留 frame 状态。

原理拆解

- generator 将控制流与迭代协议结合，是构建惰性 pipeline 的核心工具。
- 局部变量、指令位置和异常处理状态会随 generator 一起保存。
- generator 并非自动异步；普通 generator 仍在调用线程中执行。

最小可运行实验

```
def read_numbers():
    for i in range(3):
        yield i

g = read_numbers()
print(next(g))
```

易错点 / GOTCHAS

- 认为调用 generator function 会立刻执行全部函数体。
- 把 generator 与 async coroutine 混为一谈。

面试官连续追问

1. generator return value 去哪里?
2. generator frame 什么时候释放?

> **举一反三** 生成器的优势来自“按需执行 + 保存状态”，不是神秘的内存魔法。

Q44 为什么 generator 通常更省内存?

难度：★★ 频率：极高 关键词：lazy evaluation, streaming, memory

30-60 秒面试标准回答

generator 不必先物化完整结果集合，而是在消费时逐个产生元素，因此适合大文件、无限序列和流水线。

原理拆解

- 空间优势取决于下游是否也流式；最终 `list(generator)` 仍会把所有结果装进内存。
- 惰性可能推迟异常发生，使错误位置更远离 generator 创建点。
- 链式 generator 可以建立 backpressure 风格的同步流，但要关注每步缓存。

最小可运行实验

```
squares = (x * x for x in range(10_000_000))
for x in squares:
    consume(x)
```

易错点 / GOTCHAS

- 创建 generator 后立即 list()，却声称“节省内存”。
- 在 generator 内维护巨大缓存，抵消惰性优势。

面试官连续追问

1. 什么时候 list comprehension 更合适?
2. itertools 如何组合 lazy pipeline?

> **举一反三** 性能问题不能只看“是否 generator”，还要看整条数据链是否物化。

Q45 generator.send() 有什么语义?

难度：★★★★ 频率：中 关键词：send, generator protocol, coroutine history

30-60 秒面试标准回答

`send(value)` 会恢复 generator，并让当前 `yield` 表达式求值为 value；第一次启动通常必须发送 None 或用 next。

原理拆解

- yield 同时可作为“向外产生值”和“从外接收值”的暂停点。
- 这种协议是 Python 协程历史演进的一部分；现代异步 I/O 通常使用 async/await。
- 复杂 send/throw/close 协议可读性较低，业务代码中应谨慎。

最小可运行实验

```
def accumulator():
    total = 0
    while True:
        x = yield total
        total += x

g = accumulator()
next(g)
print(g.send(5))  # 5
```

易错点 / GOTCHAS

- 第一次直接 send(non-None)。
- 把 send 当成多线程消息队列。

面试官连续追问

1. throw/close 如何与 generator 交互?
2. yield expression 与 yield statement 的区别?

> **举一反三** 了解 send 有助于理解 generator delegation，但现代并发面试重点应转向 async/await。

Q46 yield from 不只是语法糖吗?

难度：★★★★ 频率：中 关键词：yield from, delegation, generator

30-60 秒面试标准回答

`yield from subiter` 不仅转发值，还会在 generator protocol 下转发 send/throw/close，并能取得子 generator 的 return value。

原理拆解

- 它把外层调用者与子 generator 之间的协议委托标准化。
- 对子 generator 的 StopIteration.value 会转化为 `yield from` 表达式的结果。
- 普通 iterable 也可用于 yield from，但高级委托能力主要体现在 generator 间。

最小可运行实验

```
def child():
    yield 1
    return 99

def parent():
    result = yield from child()
    print('child returned', result)
```

易错点 / GOTCHAS

- 只说“等价于 for x in child: yield x”。
- 不知道 return value 可通过 yield from 获取。

面试官连续追问

1. PEP 380 解决了哪些 generator delegation 问题?
2. async/await 与 yield from 的历史关系?

> **举一反三** 回答“值转发 + 协议转发 + return value”即可体现深度。

Q47 为什么 generator/iterator 通常只能消费一次?

难度：★★ 频率：高 关键词：stateful iterator, exhaustion, API

30-60 秒面试标准回答

iterator 保存当前位置，走到末尾后进入耗尽状态；再次遍历同一个 iterator 不会自动重置。

原理拆解

- iterable 可以每次产生新的 iterator，因此容器可多次遍历。
- 一次性 stream 的 API 应避免调用者误以为可重复消费。
- 需要重复消费时可重新创建 generator、物化结果，或用 itertools.tee（注意缓存成本）。

最小可运行实验

```
g = (x for x in range(3))
print(list(g))
print(list(g))  # []
```

易错点 / GOTCHAS

- 把 generator 对象当作可重复 sequence。
- 用 tee 分叉无限/速率不一致流而忽略内存缓存。

面试官连续追问

1. 如何设计“可重复遍历”的 generator factory?
2. iterator 与 iterator factory 的接口如何命名?

> **举一反三** 一次性语义要体现在 API 边界，否则最容易出现隐蔽空结果 bug。

Q48 context manager 的核心协议是什么?

难度：★★ 频率：极高 关键词：context manager, __enter__, __exit__

30-60 秒面试标准回答

`with` 通过 `__enter__` 获取资源/上下文，并保证在离开代码块时调用 `__exit__`，从而把资源释放与异常路径统一管理。

原理拆解

- 核心价值是 deterministic cleanup，而不仅仅是“代码更短”。
- `__exit__` 可以看到异常类型/值/traceback，并通过返回 truthy 表示抑制异常。
- 锁、文件、事务、临时目录、追踪 span 都可建模为上下文资源。

最小可运行实验

```
class LockCtx:
    def __enter__(self):
        self.lock.acquire(); return self
    def __exit__(self, exc_type, exc, tb):
        self.lock.release()
```

易错点 / GOTCHAS

- 在 __exit__ 中错误返回 True，吞掉不该吞的异常。
- 资源获取成功一半就异常时没有考虑清理。

面试官连续追问

1. `with` 大致如何展开成 try/finally?
2. async context manager 使用什么协议?

> **举一反三** 资源生命周期是系统可靠性的核心，context manager 是 Python 的统一抽象。

Q49 with 为什么比手写 try/finally 更好?

难度：★★ 频率：高 关键词：RAII-like, resource management, API

30-60 秒面试标准回答

`with` 本质上仍依赖异常安全清理，但它把 acquire/release 封装为可复用协议，使调用方不能轻易漏掉释放路径。

原理拆解

- try/finally 是底层控制流能力；context manager 是更高层的资源抽象。
- 组合多个资源可以用嵌套 with 或 ExitStack。
- 良好的 context manager 还可以表达事务提交/回滚等领域语义。

最小可运行实验

```
from contextlib import ExitStack
with ExitStack() as stack:
    files = [stack.enter_context(open(p)) for p in paths]
    process(files)
```

易错点 / GOTCHAS

- 说 with 比 try/finally“更安全”却解释不出封装优势。
- 过度隐藏重要事务语义，让提交/回滚难以观察。

面试官连续追问

1. ExitStack 解决什么问题?
2. context manager 如何处理部分初始化失败?

> **举一反三** 大量动态资源时 ExitStack 是比手写嵌套 try/finally 更专业的答案。

Q50 contextlib.contextmanager 是如何把 generator 变成上下文管理器的?

难度：★★★ 频率：中 关键词：contextlib, generator, cleanup

30-60 秒面试标准回答

用 `@contextmanager` 时，yield 前代码对应进入阶段，yield 值成为 `as` 绑定值，yield 后代码在退出阶段执行；通常配合 try/finally 保证清理。

原理拆解

- 函数必须形成单次 yield 的上下文语义。
- 异常会被注入 generator，使 finally/except 可以处理退出逻辑。
- 异步版本可使用 `asynccontextmanager`。

最小可运行实验

```
from contextlib import contextmanager

@contextmanager
def transaction(db):
    tx = db.begin()
    try:
        yield tx
        tx.commit()
    except:
        tx.rollback()
        raise
```

易错点 / GOTCHAS

- yield 多次破坏上下文语义。
- rollback 后忘记 re-raise，静默吞异常。

面试官连续追问

1. asynccontextmanager 与它的差异?
2. 事务嵌套/保存点如何设计?

> **举一反三** 这题可把 generator、异常传播和资源管理三个知识点串起来。

CHAPTER 06

CPython、内存、GC 与 Import

区分语言规范与 CPython 实现，理解引用计数、循环 GC、分配器与导入系统。

本章目标：能够区分 Python 语言语义和 CPython 实现，并用证据分析内存/import。

本章题目

Q51 CPython 如何管理对象生命周期？ 难度 ★★★★ · 频率 极高 · reference counting, GC, CPython

Q52 del x 是否等于删除对象？ 难度 ★★ · 频率 极高 · del, binding, refcount

Q53 什么是 reference cycle？循环 GC 为什么需要它？ 难度 ★★★ · 频率 高 · cycle, gc, reachability

Q54 weakref 适合解决什么问题？ 难度 ★★★ · 频率 中 · weakref, cache, ownership

Q55 为什么对象释放后进程 RSS 不一定下降？ 难度 ★★★★ · 频率 中 · allocator, RSS, memory

Q56 sys.getsizeof 为什么不是对象“总内存”？ 难度 ★★ · 频率 高 · getsizeof, shallow size, memory

Q57 Python import 的核心流程是什么？ 难度 ★★★ · 频率 高 · import, sys.modules, module spec

Q58 circular import 为什么会产生 partially initialized module？ 难度 ★★★ · 频率 极高 · circular import, module initialization, architecture

Q59 .pyc 是什么？什么时候会失效？ 难度 ★★ · 频率 中 · pyc, bytecode cache, import

Q60 monkey patching 什么时候有用，为什么危险？ 难度 ★★★ · 频率 中 · monkey patch, dynamic runtime, testing

Q51 CPython 如何管理对象生命周期?

难度：★★★★ 频率：极高 关键词：reference counting, GC, CPython

30-60 秒面试标准回答

CPython 以引用计数作为主要对象生命周期机制，并辅以循环垃圾收集器处理不可达引用环。语言层只保证对象最终语义，不应把所有实现细节推广到其它 Python 实现。

原理拆解

- 引用计数归零时，CPython 通常可立即销毁对象，这使资源行为看起来很确定，但不应替代显式 context manager。
- 循环 GC 关注容器对象之间的引用环；“GC”不是所有对象释放的唯一机制。
- free-threaded CPython 对引用计数实现做了大量并发适配，因此面试应区分概念与内部优化细节。

最小可运行实验

```
import gc, sys
x = []
print(sys.getrefcount(x))  # 注意调用本身会临时增加引用
print(gc.is_tracked(x))
```

易错点 / GOTCHAS

- 说“Python 用垃圾回收，所以不用关心资源释放”。
- 把 CPython refcount 当成 Python 语言规范。

面试官连续追问

1. 为什么文件仍推荐 with?
2. 循环 GC 的 generations/策略在不同版本如何演进?

> **举一反三** 资源管理依赖显式协议，内存对象生命周期才交给 runtime。

Q52 del x 是否等于删除对象?

难度：★★ 频率：极高 关键词：del, binding, refcount

30-60 秒面试标准回答

`del x` 删除名字绑定（或容器中的一个引用位置）；对象是否销毁取决于是否还有其它引用以及实现的生命周期管理。

原理拆解

- `a = obj; b = obj; del a` 后 b 仍然持有对象。
- `del container[i]` 修改的是容器内容，不是向 runtime 发“强制释放对象”的命令。
- 需要显式释放外部资源时使用 close/context manager，而不是依赖 del。

最小可运行实验

```
a = []
b = a
del a
print(b)  # 对象仍在
```

易错点 / GOTCHAS

- 把 del 当 C++ delete/free。
- 用 del 试图“修复”内存泄漏，却忽略其它引用链。

面试官连续追问

1. `__del__` 与 del 语句有什么关系?
2. 如何定位谁还引用对象?

> **举一反三** 名字、引用、对象三者分开理解，这题就不会错。

Q53 什么是 reference cycle？循环 GC 为什么需要它？

难度：★★★ 频率：高 关键词：cycle, gc, reachability

30-60 秒面试标准回答

两个或更多对象可以形成互相引用的环，使它们的 refcount 都大于 0；若整个环已从根对象不可达，循环 GC 负责识别并回收。

原理拆解

- 引用计数只能看局部“还有多少引用”，无法单独判断这些引用是否都来自一个不可达环。
- 弱引用可用于某些 observer/parent-pointer 关系减少所有权环。
- 带 finalizer 的循环在现代 CPython 中比早期版本处理更完善，但资源清理仍不应依赖 GC 时机。

最小可运行实验

```
a = []
b = []
a.append(b)
b.append(a)
del a, b
# cyclic GC 可发现不可达环
```

易错点 / GOTCHAS

- 认为 refcount 可以自动解决所有循环。
- 为了避免环而把所有引用都改成 weakref。

面试官连续追问

1. gc.collect() 什么时候有意义？
2. weakref 的生命周期回调如何使用？

> **举一反三** 是否形成“所有权”应比是否形成“引用”更重要，weakref 是表达非所有权关系的工具。

Q54 weakref 适合解决什么问题?

难度：★★★ 频率：中 关键词：weakref, cache, ownership

30-60 秒面试标准回答

weak reference 不阻止目标对象被回收，适合缓存、注册表、observer 或 parent pointer 等“不拥有对象”的关系。

原理拆解

- WeakValueDictionary/WeakKeyDictionary 可以让缓存随对象真实生命周期自动消退。
- 不是所有对象都支持弱引用；slots 类需要相应支持。
- weakref callback 不应承担复杂清理逻辑，因为执行时机与环境需要谨慎。

最小可运行实验

```
import weakref
class Item: pass
x = Item()
r = weakref.ref(x)
assert r() is x
del x
# r() 之后可能为 None
```

易错点 / GOTCHAS

- 用 weakref 解决本应由明确 ownership 管理的问题。
- 每次 r() 后不检查目标是否仍存在。

面试官连续追问

1. WeakSet 有什么用途?
2. 缓存何时应该 weak，何时应该 strong?

> **举一反三** weakref 的关键词不是“防内存泄漏”，而是“表达非所有权引用”。

Q55 为什么对象释放后进程 RSS 不一定下降?

难度：★★★★ 频率：中 关键词：allocator, RSS, memory

30-60 秒面试标准回答

Python/CPython 及底层 malloc 会缓存和复用已释放内存块；对象逻辑上死亡不意味着对应页立刻返还操作系统。因此 RSS 高不自动等于活对象泄漏。

原理拆解

- 分析内存要区分 Python object footprint、allocator retained memory、native library memory 与 OS RSS。
- 大量小对象的内存分配策略旨在提高吞吐，可能造成“释放后仍留在进程”的现象。
- 真正定位应使用 tracemalloc、对象计数、native profiler 等证据，而不是只盯 top。

最小可运行实验

```
import tracemalloc
tracemalloc.start()
# run workload
snapshot = tracemalloc.take_snapshot()
```

易错点 / GOTCHAS

- 看到 RSS 不下降就断言 leak。
- 频繁 gc.collect() 期望把所有 RSS 还给 OS。

面试官连续追问

1. 内存碎片是什么?
2. 如何区分 Python heap 与 NumPy/native memory?

> **举一反三** 性能面试里“先分类内存来源”比背 allocator 名词更有价值。

Q56 sys.getsizeof 为什么不是对象“总内存”？

难度：★★ 频率：高 关键词：getsizeof, shallow size, memory

30-60 秒面试标准回答

`sys.getsizeof(obj)` 主要报告对象自身的浅层大小，不递归统计它引用的其它对象，也不等于进程实际 RSS 增量。

原理拆解

- 容器保存引用，其元素对象可能被多个容器共享，所谓“deep size”需要定义 ownership 才能合理统计。
- interning、allocator overhead、共享对象让简单求和也可能重复计数。
- 大对象分析应结合专门工具和业务结构。

最小可运行实验

```
import sys
xs = [b'x' * 1000 for _ in range(100)]
print(sys.getsizeof(xs))  # 主要是 list 自身，不含所有 bytes 内容
```

易错点 / GOTCHAS

- 把 getsizeof(list) 当成整个数据集内存。
- 递归相加却不处理共享引用，导致重复计算。

面试官连续追问

1. 如何实现近似 deep size?
2. tracemalloc 与 getsizeof 的视角有何不同?

> **举一反三** 内存数字必须带口径：shallow、owned deep、allocated、RSS 不能混用。

Q57 Python import 的核心流程是什么?

难度：★★★ 频率：高 关键词：import, sys.modules, module spec

30-60 秒面试标准回答

import 会先检查 `sys.modules` 缓存，再通过 finder/loader 体系定位 module spec、创建模块对象并执行模块顶层代码，最后复用缓存。

原理拆解

- 模块对象在执行完成前就会进入 sys.modules，这也是 circular import 出现“部分初始化模块”的原因。
- import 不是简单“读文件”；package、namespace package、自定义 import hook 都走统一机制。
- 模块顶层代码是导入副作用，应保持轻量可预测。

最小可运行实验

```
import sys
import json
assert sys.modules['json'] is json
```

易错点 / GOTCHAS

- 认为每次 import 都重新执行模块。
- 在 module import 时启动重型网络连接/线程，制造隐藏副作用。

面试官连续追问

1. meta_path 是什么?
2. `importlib.reload` 有哪些风险?

> **举一反三** 大型工程要把“导入时间”当启动性能与依赖结构的一部分。

Q58 circular import 为什么会产生 partially initialized module?

难度：★★★ 频率：极高 关键词：circular import, module initialization, architecture

30-60 秒面试标准回答

A 执行到 import B 后暂停；B 再 import A 时，A 已在 sys.modules 中但顶层代码尚未执行完，因此 B 可能访问到尚未定义的 A 属性。

原理拆解

- 局部 import 有时能打破初始化时序，但常只是缓解症状。
- 更根本的修复是提取共同依赖、反转依赖、拆分领域边界。
- TYPE_CHECKING 条件可用于只服务静态类型的导入，避免某些 runtime 环。

最小可运行实验

```
# a.py: from b import B
# b.py: from a import A
# 两个模块在初始化阶段互相要求对方已经完成定义
```

易错点 / GOTCHAS

- 把所有 import 移到函数体当万能修复。
- 用 try/except ImportError 吞掉架构问题。

面试官连续追问

1. 包的 `__init__.py` 为什么容易制造隐式环?
2. 类型注解如何减少 runtime import?

> **举一反三** 循环导入经常是模块边界设计问题，而不是 import 语法问题。

Q59 .pyc 是什么？什么时候会失效？

难度：★★ 频率：中 关键词：pyc, bytecode cache, import

30-60 秒面试标准回答

.pyc 是编译后的 Python bytecode/code object 缓存，用于减少后续导入的编译成本；它不是 native machine code，也不应视为稳定跨版本分发格式。

原理拆解

- 缓存有效性由解释器版本与源文件校验策略等共同决定。
- 删除 pyc 通常只影响下一次重新编译，不会改变语言语义。
- 部署时“只有 pyc”并不等于源码级安全保护。

最小可运行实验

```
# 常见位置：__pycache__/module.cpython-314.pyc
# 具体 tag 随解释器版本/实现变化
```

易错点 / GOTCHAS

- 把 pyc 当成机器码加密保护。
- 把不同 Python 版本的 pyc 当稳定 ABI。

面试官连续追问

1. hash-based pyc 与 timestamp-based pyc 有何差异?
2. 为何容器构建要关注 pycache?

> **举一反三** 回答 pyc 时强调“缓存，不是安全/跨版本发布格式”。

Q60 monkey patching 什么时候有用，为什么危险?

难度：★★★ 频率：中 关键词：monkey patch, dynamic runtime, testing

30-60 秒面试标准回答

monkey patching 是运行时替换模块/类/对象属性。测试、兼容层、instrumentation 偶尔有用，但它制造隐式全局状态、顺序依赖与调试困难。

原理拆解

- patch 的关键是“在被查找的位置 patch”，而不一定是定义位置。
- 测试框架通常用 context/fixture 自动恢复 patch，避免污染其它测试。
- 生产热补丁必须有明确版本约束、回滚与监控。

最小可运行实验

```
from unittest.mock import patch

with patch('myapp.service.time.time', return_value=0):
    assert deterministic_result()
```

易错点 / GOTCHAS

- patch 错命名空间，测试仍调用原对象。
- 测试结束后不恢复全局 patch。

面试官连续追问

1. 为什么依赖注入通常比 monkey patch 更可维护?
2. patch class method 与 instance attribute 有何差异?

> **举一反三** monkey patch 是能力，不是架构；可测试设计优先显式依赖注入。

CHAPTER 07

线程、进程、GIL 与多核并行

掌握经典 GIL 模型，并覆盖 Python 3.14 的 free-threaded 与 InterpreterPoolExecutor。

本章目标：能够根据 workload 选择线程/进程/free-threaded/subinterpreter 并讨论同步成本。

本章题目

Q61 GIL 是什么？如何用现代版本回答？ 难度 ★★★ · 频率 极高 · GIL, threads, CPython

Q62 “有 GIL 所以不需要锁”为什么是错的？ 难度 ★★★ · 频率 极高 · race condition, lock, atomicity

Q63 Python 3.14 free-threaded CPython 改变了什么？ 难度 ★★★★★ · 频率 高 · free-threading, PEP703, Python3.14

Q64 Thread vs Process 应该怎么选？ 难度 ★★★ · 频率 极高 · thread, process, workload

Q65 多进程的主要代价是什么？ 难度 ★★ · 频率 高 · multiprocessing, IPC, isolation

Q66 multiprocessing 为什么经常出现 pickle error？ 难度 ★★★ · 频率 高 · pickle, serialization, process pool

Q67 ThreadPoolExecutor 与 ProcessPoolExecutor 的关键差异？ 难度 ★★ · 频率 极高 · concurrent.futures, executor, future

Q68 fork、spawn、forkserver 有什么区别？ 难度 ★★★★ · 频率 高 · multiprocessing start method, fork, spawn

Q69 什么是 race condition？如何系统治理？ 难度 ★★ · 频率 极高 · race, synchronization, concurrency

Q70 Python 3.14 InterpreterPoolExecutor 是什么？ 难度 ★★★★★ · 频率 高 · InterpreterPoolExecutor, subinterpreters, Python3.14

Q61 GIL 是什么？如何用现代版本回答？

难度：★★★ 频率：极高 关键词：GIL, threads, CPython

30-60 秒面试标准回答

在经典/默认 CPython 中，GIL 限制同一解释器中通常只有一个线程同时执行 Python 代码，因此 CPU-bound 纯 Python 线程难以线性利用多核；I/O 阻塞和许多 native 扩展可释放 GIL。Python 3.13+ 另有可选 free-threaded build。

原理拆解

- GIL 是 CPython 实现机制，不是 Python 语言规范。
- I/O-bound 线程有价值，因为线程等待 I/O 时其它线程可运行。
- “有 GIL”不等于任何 Python 操作都线程安全，也不等于 native numeric library 不能多核并行。

最小可运行实验

```
import threading
# 对 CPU-bound Python 循环，默认 CPython 多线程通常不会带来线性多核加速
# 对网络/文件等待，threads 仍可能显著提高并发度
```

易错点 / GOTCHAS

- 回答停留在“Python 有 GIL，所以线程没用”。
- 把 GIL 当成业务数据锁。

面试官连续追问

1. C 扩展何时释放 GIL?
2. free-threaded build 对这套结论改变什么?

举一反三 2026 面试必须主动补充 free-threaded CPython 的版本背景。

2026 版本注释 默认 CPython 与 free-threaded build 必须分开回答。Python 3.13+ 可选 free-threaded；Python 3.14 仍不是默认无 GIL。

Q62 “有 GIL 所以不需要锁”为什么是错的?

难度：★★★ 频率：极高 关键词：race condition, lock, atomicity

30-60 秒面试标准回答

GIL 只约束解释器执行，不保证由多个字节码/业务步骤组成的不变量原子成立；共享可变状态仍可能发生 check-then-act、lost update 等竞态。

原理拆解

- `if key not in d: d[key] = create()` 是多步骤逻辑，即使单个内置操作在某版本看似原子，也不能据此推导业务事务原子。
- free-threaded 构建进一步要求代码显式面对真正并行。
- 最好策略往往是减少共享可变状态，使用 Queue/message passing 或明确锁粒度。

最小可运行实验

```
lock = threading.Lock()
with lock:
    if key not in cache:
        cache[key] = build()
```

易错点 / GOTCHAS

- 依赖“某个 CPython 内置操作现在恰好原子”作为程序正确性协议。
- 锁范围过大把并发程序退化成串行。

面试官连续追问

1. RLock 与 Lock 何时选择?
2. 如何避免 deadlock?

> **举一反三** 并发正确性要围绕业务不变量设计，而不是围绕 GIL 猜测。

Q63 Python 3.14 free-threaded CPython 改变了什么?

难度：★★★★★ 频率：高 关键词：free-threading, PEP703, Python3.14

30-60 秒面试标准回答

Python 3.13 起提供禁用 GIL 的 free-threaded build；Python 3.14 进入更正式的支持阶段，但仍不是默认构建。它允许线程在多核上并行执行 Python 代码，但第三方扩展兼容性与线程安全要求更高。

原理拆解

- 可用 `sys._is_gil_enabled()` 等方式判断运行进程是否启用 GIL（以当前版本官方文档为准）。
- 某些不支持 free-threading 的 extension module 可能让运行时重新启用 GIL。
- 过去依赖 GIL 获得的“偶然线程安全”必须重新审视，库作者需明确同步策略。

最小可运行实验

```
import sys
if hasattr(sys, '_is_gil_enabled'):
    print('GIL enabled:', sys._is_gil_enabled())
```

易错点 / GOTCHAS

- 说 Python 3.14 已经默认移除 GIL。
- 认为现有所有 C 扩展都无需修改即可安全并行。

面试官连续追问

1. free-threaded 的性能一定更好吗?
2. 对象模型/引用计数为并行做了哪些方向的改变?

举一反三 面试标准句：可选、真实并行、生态兼容性仍是关键约束。

2026 版本注释 PEP 779 使 free-threaded Python 在 3.14 进入更正式的支持阶段；是否启用仍依赖构建/运行环境。

Q64 Thread vs Process 应该怎么选?

难度：★★★ 频率：极高 关键词：thread, process, workload

30-60 秒面试标准回答

默认 CPython 下，I/O-bound 常优先线程/async；CPU-bound 纯 Python 常优先进程。最终选择还要结合序列化成本、内存隔离、共享状态、native code、free-threaded 构建等条件。

原理拆解

- threads 共享地址空间，通信便宜但同步复杂。
- process 提供隔离和多核，但 startup/IPC/pickle 成本更高。
- CPU-heavy NumPy 等 native 库可能已在 C/BLAS 内并行，外层再开进程可能过度订阅。

最小可运行实验

```
# I/O
ThreadPoolExecutor(...)
# CPU pure Python (默认 CPython)
ProcessPoolExecutor(...)
# 仍需 benchmark
```

易错点 / GOTCHAS

- 背“IO=线程，CPU=进程”而不问环境。
- 忽略任务粒度，小任务被 IPC 开销淹没。

面试官连续追问

1. 混合 I/O + CPU pipeline 如何设计?
2. 如何控制进程池任务粒度?

> **举一反三** 并发模型选择是成本模型题，不是口诀题。

Q65 多进程的主要代价是什么?

难度：★★ 频率：高 关键词：multiprocessing, IPC, isolation

30-60 秒面试标准回答

进程带来独立地址空间和独立解释器，换来更强隔离与多核能力；代价包括启动、序列化、IPC、额外内存和运维复杂度。

原理拆解

- fork 的 copy-on-write 并不意味着“内存完全免费”；写入后页面会复制。
- spawn 需要重新导入/初始化，重型模块会放大启动成本。
- 大量小任务频繁跨进程传输对象会很低效。

最小可运行实验

```
# 把小任务批成 chunk 往往比逐条 submit 更高效
for chunk in chunks(data, 1000):
    pool.submit(process_chunk, chunk)
```

易错点 / GOTCHAS

- 只关注绕过 GIL，不谈 IPC 和内存。
- 把数据库连接/socket 直接当作可安全跨进程对象共享。

面试官连续追问

1. 共享内存什么时候值得用?
2. fork 后连接池为什么危险?

> **举一反三** 进程是隔离边界，不仅是“性能工具”。

Q66 multiprocessing 为什么经常出现 pickle error?

难度：★★★ 频率：高 关键词：pickle, serialization, process pool

30-60 秒面试标准回答

进程池通常需要把 callable、参数和结果序列化传给 worker；lambda、局部函数、打开的资源、某些 C 扩展对象等并不适合 pickle。

原理拆解

- 顶层可导入函数是最稳妥的 worker callable。
- 在 spawn/forkserver 场景，`__main__` 可导入性和 `if __name__ == "__main__"` 更重要。
- 能 pickle 不代表传输高效：巨大 DataFrame 复制成本可能成为真正瓶颈。

最小可运行实验

```
def worker(x):
    return x * x

if __name__ == '__main__':
    with ProcessPoolExecutor() as ex:
        print(list(ex.map(worker, range(10))))
```

易错点 / GOTCHAS

- 把本地 lambda 提交到进程池并期望所有平台工作。
- 只修 pickle error，不评估序列化体积。

面试官连续追问

1. cloudpickle 能解决什么、不能解决什么?
2. 如何减少大对象跨进程复制?

> **举一反三** 序列化兼容与序列化成本是两个不同问题。

Q67 ThreadPoolExecutor 与 ProcessPoolExecutor 的关键差异?

难度：★★ 频率：极高 关键词：concurrent.futures, executor, future

30-60 秒面试标准回答

两者共享 Executor/Future 抽象，但一个在同进程线程中执行、一个在独立进程中执行；因此共享内存、GIL、失败隔离和序列化约束不同。

原理拆解

- 统一 API 让代码结构相似，但并不意味着可无成本互换。
- Future.result() 在 worker 内错误等待同池 Future 可能形成死锁。
- shutdown、异常传播、取消语义都应显式理解。

最小可运行实验

```
with ThreadPoolExecutor(max_workers=8) as ex:
    futures = [ex.submit(fetch, u) for u in urls]
    for f in futures:
        handle(f.result())
```

易错点 / GOTCHAS

- 在单 worker pool 中任务等待同池另一个任务。
- 无限 submit 导致任务队列和内存膨胀。

面试官连续追问

1. as_completed 与 map 的差异?
2. 如何实现 bounded submission?

> **举一反三** 高级用法关注背压，而不是一次性提交百万个 Future。

Q68 fork、spawn、forkserver 有什么区别?

难度：★★★★ 频率：高 关键词：multiprocessing start method, fork, spawn

30-60 秒面试标准回答

三者定义 worker 进程如何启动：fork 复制当前进程状态，spawn 启动干净解释器并重新导入，forkserver 由专门服务进程负责 fork。不同平台/版本默认值不同。

原理拆解

- fork 对多线程父进程存在锁状态/库状态风险。
- spawn 更干净但启动和 import 成本高。
- Python 3.14 在 POSIX 上的默认策略已发生重要变化，应以当前官方文档为准，而不是依赖旧经验。

最小可运行实验

```
import multiprocessing as mp
print(mp.get_start_method())
# 必要时显式选择 context，而不是假定全平台相同
```

易错点 / GOTCHAS

- 写死“Linux 默认永远是 fork”。
- fork 后复用线程池、数据库连接而不重新初始化。

面试官连续追问

1. 为什么深度学习框架常对 fork 有特殊建议?
2. mp_context 参数如何传给 ProcessPoolExecutor?

举一反三 版本敏感题要明确“平台 + Python 版本”。

2026 版本注释 multiprocessing 默认 start method 随平台与版本变化。不要把旧版 Linux 的 fork 经验写成永恒结论。

Q69 什么是 race condition？如何系统治理？

难度：★★ 频率：极高 关键词：race, synchronization, concurrency

30-60 秒面试标准回答

race condition 指结果依赖并发执行时序且违反预期不变量。治理不只靠锁，还可以通过 immutable data、消息传递、原子数据库操作、partition ownership 等减少共享状态。

原理拆解

- 先明确需要保护的 invariant，再确定 critical section。
- 锁解决互斥但引入 deadlock/convoy/contention 风险。
- 分布式系统中的 race 可能跨进程/机器，Python Lock 无法解决，需要数据库事务、CAS、幂等键等。

最小可运行实验

```
# check-then-act race
if order.status == 'pending':
    charge(order)
    order.status = 'paid'
# 多 worker 下需要更强的一致性机制
```

易错点 / GOTCHAS

- 只在 Python 代码外层加 Lock，却服务部署有多个进程/实例。
- 锁住整个请求处理路径。

面试官连续追问

1. lost update、TOCTOU 分别是什么？
2. 数据库唯一约束如何作为并发安全最后防线？

> **举一反三** 真正成熟的答案会把线程 race 延伸到多进程/分布式一致性。

Q70 Python 3.14 InterpreterPoolExecutor 是什么?

难度：★★★★★ 频率：高 关键词：InterpreterPoolExecutor, subinterpreters, Python3.14

30-60 秒面试标准回答

InterpreterPoolExecutor 在多个线程中运行彼此隔离的解释器；每个解释器有自己的运行时状态和 GIL，因此可实现真正多核执行，同时避免完整 OS 进程的某些成本。

原理拆解

- 解释器彼此隔离，mutable object 不能像普通线程那样直接共享。
- 任务与结果仍需要可传输/序列化，当前使用上必须认真评估数据交换成本。
- 它提供 thread/process 之外的新并行选择，但生态和工具成熟度仍需结合具体版本验证。

最小可运行实验

```
from concurrent.futures import InterpreterPoolExecutor

def cpu(x):
    return sum(i * i for i in range(x))

with InterpreterPoolExecutor() as ex:
    print(list(ex.map(cpu, [10000, 20000])))
```

易错点 / GOTCHAS

- 把它当作“没有隔离的 ThreadPool”。
- 以为可以直接共享普通 mutable object。

面试官连续追问

1. 与 ProcessPoolExecutor 的失败隔离和启动成本如何比较?
2. PEP 734 等跨解释器通信方向解决什么问题?

举一反三 这是 2026 Python 高级面试中非常有区分度的新题。

2026 版本注释 InterpreterPoolExecutor 在 Python 3.14 加入 concurrent.futures，核心特征是隔离解释器 + 各自 GIL + 多核并行。

CHAPTER 08

asyncio 与结构化并发

从 event loop 到 TaskGroup、取消传播、限流、背压与生产级 worker。

本章目标：能够写出具备取消、限流、背压和优雅关闭的 async 代码。

本章题目

Q71 concurrency 与 parallelism 有什么区别？ 难度 ★ · 频率 极高 · concurrency, parallelism, asyncio

Q72 coroutine、Task、Future 分别是什么？ 难度 ★★★ · 频率 极高 · coroutine, Task, Future

Q73 await 的核心语义是什么？ 难度 ★★ · 频率 极高 · await, suspension, event loop

Q74 为什么 async 函数里 time.sleep 是严重问题？ 难度 ★ · 频率 极高 · blocking I/O, event loop, sleep

Q75 asyncio.gather 与 TaskGroup 怎么选？ 难度 ★★★★ · 频率 极高 · TaskGroup, gather, structured concurrency

Q76 为什么不能随便吞 asyncio.CancelledError？ 难度 ★★★★ · 频率 极高 · cancellation, CancelledError, cleanup

Q77 async code 中如何调用阻塞函数？ 难度 ★★ · 频率 极高 · to_thread, executor, blocking

Q78 async 能解决 CPU-bound 吗？ 难度 ★★ · 频率 极高 · CPU-bound, event loop, offload

Q79 Semaphore 在 async 服务中解决什么问题？ 难度 ★★ · 频率 高 · Semaphore, rate limit, bounded concurrency

Q80 如何设计一个生产级 asyncio worker？ 难度 ★★★★★ · 频率 极高 · worker, backpressure, graceful shutdown

Q71 concurrency 与 parallelism 有什么区别?

难度：★ 频率：极高 关键词：concurrency, parallelism, asyncio

30-60 秒面试标准回答

concurrency 是多个任务在一段时间内交错推进；parallelism 是多个任务在同一时刻真正同时执行。asyncio 主要解决并发，是否并行取决于底层线程/进程/解释器/native code。

原理拆解

- 单线程 event loop 可以高度并发 I/O，但仍不是多核 CPU 并行。
- 多线程在经典 GIL CPython 下能并发 I/O；free-threaded 构建下可进一步并行 Python code。
- 术语明确后，架构选择更容易。

最小可运行实验

```
# asyncio: concurrency
# process/free-threaded/subinterpreters/native parallelism: possible parallelism
```

易错点 / GOTCHAS

- 把 async 等同于 parallel。
- 看到多线程就断言一定并行。

面试官连续追问

1. 异步 I/O 为什么在单线程仍能高吞吐?
2. 并行会自动提高吞吐吗?

> **举一反三** 先分类“等待型”与“计算型”工作负载。

Q72 coroutine、Task、Future 分别是什么?

难度：★★★ 频率：极高 关键词：coroutine, Task, Future

30-60 秒面试标准回答

调用 async function 得到 coroutine object；Task 把 coroutine 调度到 event loop 并管理其生命周期；Future 是“未来结果”的低层抽象，Task 具有 Future-like 接口。

原理拆解

- 只创建 coroutine 而不 await/schedule，它通常不会自动运行。
- Task 需要保留引用并正确收集异常，否则可能看到“Task exception was never retrieved”。
- `asyncio.Future` 与 `concurrent.futures.Future` 面向不同并发体系。

最小可运行实验

```
async def work(): return 42

async def main():
    coro = work()
    task = asyncio.create_task(coro)
    print(await task)
```

易错点 / GOTCHAS

- 调用 async function 后以为代码已经执行。
- fire-and-forget task 不保存引用、不处理异常。

面试官连续追问

1. create_task 与直接 await 的并发差异?
2. 什么时候需要低层 Future?

> **举一反三** 生产代码优先用 TaskGroup 管理一组任务，而不是散落的裸 create_task。

Q73 await 的核心语义是什么?

难度：★★ 频率：极高 关键词：await, suspension, event loop

30-60 秒面试标准回答

`await` 在等待对象未就绪时允许当前 coroutine 挂起，把控制权交回调度器；未来就绪后再恢复。它不是“把函数放到线程里”。

原理拆解

- 只有底层操作真正非阻塞/可挂起，event loop 才能在等待期间推进其它任务。
- CPU 密集循环中没有 await point，会长时间霸占 event loop。
- 频繁 await 也有调度成本，要在正确边界切换。

最小可运行实验

```
async def fetch_all(client, urls):
    tasks = [asyncio.create_task(client.get(u)) for u in urls]
    return await asyncio.gather(*tasks)
```

易错点 / GOTCHAS

- async def 内部调用阻塞库就认为非阻塞。
- 为了“异步”把纯 CPU 小函数都改成 async。

面试官连续追问

1. awaitable protocol 是什么?
2. event loop 如何获知 I/O ready?

> **举一反三** async 的性能来自等待期间让出执行权，而不是关键字本身。

Q74 为什么 async 函数里 time.sleep 是严重问题?

难度：★ 频率：极高 关键词：blocking I/O, event loop, sleep

30-60 秒面试标准回答

`time.sleep` 会阻塞运行 event loop 的线程，期间同 loop 上其它任务都无法推进；应使用 `await asyncio.sleep` 或把无法改造的阻塞调用 offload。

原理拆解

- 同理，requests、同步 DB client、同步文件/SDK 等都可能阻塞 event loop。
- 一个阻塞点即可让“异步服务”尾延迟显著恶化。
- 诊断需要 event-loop lag 指标或 profiling，而不只是看 CPU。

最小可运行实验

```
async def bad():
    time.sleep(1)

async def good():
    await asyncio.sleep(1)
```

易错点 / GOTCHAS

- 只检查 endpoint 定义是不是 async def。
- 把 blocking function 包成 async def，却没改变内部实现。

面试官连续追问

1. 如何发现 event loop blocking?
2. asyncio.to_thread 适合什么场景?

> **举一反三** FastAPI 性能问题第一排查项之一就是阻塞调用进入 event loop。

Q75 asyncio.gather 与 TaskGroup 怎么选?

难度：★★★★ 频率：极高 关键词：TaskGroup, gather, structured concurrency

30-60 秒面试标准回答

`gather` 适合收集一组 awaitable 的结果；TaskGroup 提供 structured concurrency，把子任务生命周期绑定到 async with 作用域，并定义更可靠的失败/取消传播。

原理拆解

- TaskGroup 中一个任务异常时，通常会取消其余任务并在退出时聚合异常。
- gather 的异常/取消语义与 TaskGroup 不同，不能仅把两者看作语法替换。
- 新业务并发树通常优先 TaskGroup，因为 ownership 更清楚。

最小可运行实验

```
async with asyncio.TaskGroup() as tg:
    t1 = tg.create_task(job1())
    t2 = tg.create_task(job2())
# 离开作用域时任务已结束或异常已传播
```

易错点 / GOTCHAS

- 把 TaskGroup 当成“更快的 gather”。
- 创建子任务却让其生命周期逃逸出 owning scope。

面试官连续追问

1. ExceptionGroup 如何处理?
2. 为什么 structured concurrency 更容易清理资源?

> **举一反三** 高级回答要谈“任务所有权与生命周期”，不只谈返回值格式。

> **2026 版本注释** TaskGroup 自 Python 3.11 引入，3.13/3.14 仍有取消处理与 create_task 参数传递等细节演进。

Q76 为什么不能随便吞 asyncio.CancelledError?

难度：★★★★ 频率：极高 关键词：cancellation, CancelledError, cleanup

30-60 秒面试标准回答

取消是 asyncio 结构化并发的重要控制信号。协程应在 finally 中清理；若显式捕获 CancelledError，通常清理后要继续传播，否则 TaskGroup/timeout 等上层机制可能行为异常。

原理拆解

- cancel() 不是立即杀线程，而是在合适的挂起点向任务注入取消。
- CancelledError 在现代 asyncio 中继承 BaseException，普通 `except Exception` 通常不会吞它。
- 库代码要把 cancellation 当正常控制流的一部分，设计幂等 cleanup。

最小可运行实验

```
async def worker():
    resource = await acquire()
    try:
        await run(resource)
    finally:
        await release(resource)
```

易错点 / GOTCHAS

- `except BaseException: pass` 无意吞取消。
- cleanup 自己无限阻塞，导致服务无法优雅关闭。

面试官连续追问

1. shield 有什么用途与风险?
2. timeout 如何依赖 cancellation?

> **举一反三** 取消语义是 async 稳定性的核心，不是“异常边角料”。

> **2026 版本注释** 官方文档明确建议清理后通常继续传播 CancelledError，结构化并发组件内部依赖取消语义。

Q77 async code 中如何调用阻塞函数?

难度：★★ 频率：极高 关键词：to_thread, executor, blocking

30-60 秒面试标准回答

不能改造的阻塞 I/O 可使用 `asyncio.to_thread` 或 executor offload；CPU-heavy 工作则需要评估 GIL、进程池、free-threaded 等。

原理拆解

- to_thread 主要适合 I/O-bound 阻塞调用；在默认 CPython 下对纯 Python CPU-heavy 不提供理想多核扩展。
- 线程 offload 仍需限制并发，否则会创建排队/资源压力。
- contextvars 在现代 to_thread 中有相应传播语义，框架集成要注意。

最小可运行实验

```
result = await asyncio.to_thread(blocking_sdk_call, arg)
```

易错点 / GOTCHAS

- 所有 blocking 都无限 to_thread，没有线程池容量控制。
- CPU-heavy 任务仍在同进程大量线程里竞争。

面试官连续追问

1. run_in_executor 与 to_thread 的差异?
2. 如何把 CPU task 送 ProcessPool?

> **举一反三** offload 是兼容桥梁，不是替代真正 async client 的长期最佳方案。

Q78 async 能解决 CPU-bound 吗?

难度：★★ 频率：极高 关键词：CPU-bound, event loop, offload

30-60 秒面试标准回答

单线程 asyncio 本身不能让 CPU-heavy Python 多核并行；长计算还会阻塞 loop。应拆分/下沉到进程、free-threaded threads、subinterpreters、native extension 或独立计算服务。

原理拆解

- 偶尔 `await asyncio.sleep(0)` 只能提高公平性，不能增加总 CPU。
- 把 CPU 任务放 event loop 会同时恶化其它 I/O 请求的尾延迟。
- 架构上应为 CPU worker 设置独立容量与队列，防止拖垮 API worker。

最小可运行实验

```
async def endpoint():
    result = await loop.run_in_executor(cpu_pool, heavy_compute, payload)
    return result
```

易错点 / GOTCHAS

- 给 CPU 循环加 async 关键字就认为并行。
- 在请求线程直接执行大模型/图像处理而无隔离。

面试官连续追问

1. 什么时候应该拆独立 job queue?
2. 进程池大小如何依据 CPU/任务内存估算?

> **举一反三** 高并发 API 应把“请求编排”和“重计算”分成不同容量池。

Q79 Semaphore 在 async 服务中解决什么问题?

难度：★★ 频率：高 关键词：Semaphore, rate limit, bounded concurrency

30-60 秒面试标准回答

Semaphore 限制同时进入某个关键资源的任务数量，用于保护数据库连接、第三方 API、文件描述符和内存，而不是提高无限并发。

原理拆解

- 吞吐通常存在最佳并发区间，超过后只会增加排队、上下文切换和超时。
- Semaphore 只限制本进程范围；多实例全局限流需 Redis/网关/服务端配额等机制。
- 获取 semaphore 的等待也要纳入 timeout 与 cancellation 设计。

最小可运行实验

```
sem = asyncio.Semaphore(20)

async def call_upstream(x):
    async with sem:
        return await client.get(x)
```

易错点 / GOTCHAS

- 把本地 semaphore 当集群级 rate limiter。
- 只限并发数，不设置上游 timeout。

面试官连续追问

1. Semaphore 与 Queue 的背压差别?
2. API rate limit 与 concurrency limit 有何不同?

> **举一反三** 限制并发是稳定性机制；容量要用测量而不是拍脑袋。

Q80 如何设计一个生产级 asyncio worker?

难度：★★★★★ 频率：极高 关键词：worker, backpressure, graceful shutdown

30-60 秒面试标准回答

核心不是一次 gather 全部任务，而是有界队列、有界并发、超时、取消传播、重试策略、幂等、优雅停机和可观测性。

原理拆解

- Queue 负责生产/消费解耦和背压；worker 数量控制同时在途任务。
- 重试必须限定可重试异常并使用 backoff/jitter；消息处理要有幂等/去重策略。
- shutdown 要停止接收新任务、取消/等待在途任务、释放资源并保存必要状态。

最小可运行实验

```
async def worker(queue):
    while True:
        item = await queue.get()
        try:
            async with asyncio.timeout(10):
                await handle(item)
        finally:
            queue.task_done()
```

易错点 / GOTCHAS

- 一次性创建百万 Task。
- retry loop 无上限、无 jitter、无幂等。

面试官连续追问

1. 如何实现 dead-letter queue?
2. 如何监控 queue lag、in-flight、error rate?

> **举一反三** 这是一道把 asyncio 语法升级为可靠系统设计的综合题。

CHAPTER 09

Typing、测试、打包与工程质量

掌握现代类型系统、dataclass、测试边界、依赖隔离与 pyproject.toml。

本章目标：能够用类型、测试和打包工具降低大型代码库的维护成本。

本章题目

Q81 动态语言为什么仍需要 type hints? 难度 ★ · 频率 极高 · typing, static analysis, API

Q82 Any 和 object 有什么区别? 难度 ★★★ · 频率 高 · Any, object, type safety

Q83 Protocol 解决了什么问题? 难度 ★★★★ · 频率 高 · Protocol, structural typing, duck typing

Q84 TypeVar / 新泛型语法解决什么问题? 难度 ★★★★ · 频率 高 · generics, TypeVar, type parameter

Q85 ParamSpec 为什么适合给装饰器做 typing? 难度 ★★★★★ · 频率 中 · ParamSpec, Callable, decorator typing

Q86 dataclass 解决了什么？有哪些陷阱? 难度 ★★ · 频率 极高 · dataclass, default_factory, value object

Q87 unit test 与 integration test 如何划分? 难度 ★★ · 频率 高 · testing, test pyramid, feedback

Q88 mock 应该 mock 什么? 难度 ★★★ · 频率 极高 · mock, boundary, test design

Q89 virtual environment / lockfile 分别解决什么问题? 难度 ★★ · 频率 高 · venv, dependencies, reproducibility

Q90 pyproject.toml 在现代 Python 项目中的角色? 难度 ★★★ · 频率 高 · pyproject.toml, packaging, build

Q81 动态语言为什么仍需要 type hints?

难度：★ 频率：极高 关键词：typing, static analysis, API

30-60 秒面试标准回答

type hints 不改变 Python 动态运行本质，主要服务静态分析、IDE、重构、文档和 API 契约；默认 runtime 不会自动完整执行类型检查。

原理拆解

- 类型注解最有价值的地方是大型代码库的接口边界，而不是追求“100% 类型化”。
- 类型系统可表达 Optional、Protocol、generics、Literal、TypedDict 等复杂契约。
- runtime validation 是 Pydantic 等另一层机制，不应与 typing 混淆。

最小可运行实验

```
def parse_user(data: dict[str, object]) -> 'User':
    ...
```

易错点 / GOTCHAS

- 说“加了类型注解，Python 会拒绝错误类型调用”。
- 把 type checker 的规则当 runtime 语言行为。

面试官连续追问

1. mypy/pyright 的 strict 策略如何渐进落地?
2. runtime validation 与 static typing 如何分工?

> **举一反三** 类型注解的目标是降低变更成本和接口歧义。

Q82 Any 和 object 有什么区别?

难度：★★★ 频率：高 关键词：Any, object, type safety

30-60 秒面试标准回答

`object` 是所有对象的安全顶层类型，拿到它后只能使用所有 object 都保证的操作；`Any` 基本绕过静态类型约束，调用任意属性通常不会被检查。

原理拆解

- 从不可信边界得到数据时，使用 object/Unknown 再 narrowing 往往比 Any 更安全。
- Any 会“传播”，让一片代码失去类型检查能力。
- 迁移旧代码时 Any 有现实价值，但要把它限制在边界。

最小可运行实验

```
def use_obj(x: object) -> str:
    return str(x)

def unsafe(x: Any):
    return x.whatever.deep.call()
```

易错点 / GOTCHAS

- 把 object 当成“等价于 Any”。
- 公共 API 到处返回 Any。

面试官连续追问

1. Unknown 与 Any 在不同 type checker 中有什么区别?
2. 如何用 isinstance/TypeGuard narrowing?

> **举一反三** “Any 最小化”是渐进类型系统的实用工程原则。

Q83 Protocol 解决了什么问题?

难度：★★★★ 频率：高 关键词：Protocol, structural typing, duck typing

30-60 秒面试标准回答

Protocol 让静态类型系统表达结构化子类型：对象不必显式继承某个基类，只要提供协议要求的成员即可被接受。

原理拆解

- 它把 Python 的 duck typing 显式化为可检查接口。
- 特别适合依赖注入、文件类对象、repository/service interface。
- Protocol 只描述调用者真正需要的最小能力，可降低耦合。

最小可运行实验

```
from typing import Protocol

class Closable(Protocol):
    def close(self) -> None: ...

def shutdown(x: Closable) -> None:
    x.close()
```

易错点 / GOTCHAS

- 为每个 Protocol 都要求实现类显式继承。
- 定义巨大 Protocol，把实现再次绑死。

面试官连续追问

1. runtime_checkable 有哪些限制?
2. Protocol 与 ABC 如何取舍?

> **举一反三** 面向接口编程在 Python 中往往更适合小 Protocol，而不是深继承树。

Q84 TypeVar / 新泛型语法解决什么问题?

难度：★★★★ 频率：高 关键词：generics, TypeVar, type parameter

30-60 秒面试标准回答

泛型的价值是表达不同位置类型之间的关系，而不是简单说“可以放任何类型”。现代 Python 还支持更简洁的类型参数语法。

原理拆解

- `def first[T](xs: Sequence[T]) -> T` 表达返回类型与输入元素类型相同。
- bound/constraints 可进一步限制 T。
- variance 关系决定容器/接口子类型兼容性，是高级 typing 常见追问。

最小可运行实验

```
from collections.abc import Sequence

def first[T](xs: Sequence[T]) -> T:
    return xs[0]
```

易错点 / GOTCHAS

- 用 Any 代替所有泛型，失去类型关系。
- 把泛型当 C++ template 的运行时实例化。

面试官连续追问

1. covariant/contravariant 如何理解?
2. Self 类型解决什么问题?

> **举一反三** 泛型回答重点是“保持输入输出类型关系”。

Q85 ParamSpec 为什么适合给装饰器做 typing?

难度：★★★★★ 频率：中 关键词：ParamSpec, Callable, decorator typing

30-60 秒面试标准回答

ParamSpec 可以捕获 callable 的整个参数列表并转发给 wrapper，配合 TypeVar/返回类型避免 `Callable[..., R]` 丢失调用签名。

原理拆解

- 它特别适合 logging/retry/permission 等不改变参数签名的 decorator。
- 如果 decorator 添加/删除参数，可结合 Concatenate 等工具表达。
- runtime `functools.wraps` 与 static ParamSpec 解决的是不同层的问题。

最小可运行实验

```
from typing import Callable, ParamSpec, TypeVar
P = ParamSpec('P')
R = TypeVar('R')

def deco(fn: Callable[P, R]) -> Callable[P, R]:
    ...
```

易错点 / GOTCHAS

- 用了 wraps 就认为 type checker 会自动保留签名。
- 所有 wrapper 标成 Callable[..., Any]。

面试官连续追问

1. Concatenate 用于什么?
2. async decorator 的返回类型如何表达?

> **举一反三** 高级 typing 面试中，ParamSpec 是装饰器题的标准升级点。

Q86 dataclass 解决了什么？有哪些陷阱?

难度：★★ 频率：极高 关键词：dataclass, default_factory, value object

30-60 秒面试标准回答

dataclass 根据字段声明自动生成 init/repr/eq 等样板代码，适合数据承载和值对象；它不自动提供 runtime 类型验证。

原理拆解

- 可变默认值应使用 `field(default_factory=list)` 等工厂。
- frozen=True 提供较强不可变约束但不是绝对安全沙箱。
- slots=True、order、kw_only 等选项会改变生成行为，应基于语义使用。

最小可运行实验

```
from dataclasses import dataclass, field

@dataclass(slots=True)
class User:
    name: str
    tags: list[str] = field(default_factory=list)
```

易错点 / GOTCHAS

- 字段写 `tags: list[str] = []`。
- 认为 dataclass 会在运行时强制 name 必须是 str。

面试官连续追问

1. dataclass 与 NamedTuple/Pydantic model 如何取舍?
2. unsafe_hash 为什么叫 unsafe?

> **举一反三** 先问：它是纯数据对象、验证模型还是领域实体？不同工具职责不同。

Q87 unit test 与 integration test 如何划分?

难度：★★ 频率：高 关键词：testing, test pyramid, feedback

30-60 秒面试标准回答

unit test 关注小范围逻辑并提供快速反馈；integration test 验证组件真实协作。边界不应只按“有没有数据库”机械定义，而应看测试隔离范围和目标风险。

原理拆解

- 纯 unit test 无法发现 SQL、序列化、网络协议、配置等集成问题。
- 全部用 integration/e2e 又会导致反馈慢、故障定位难。
- 健康测试组合按风险配置：核心纯逻辑大量快测，关键边界少量真实集成，用户路径保留端到端。

最小可运行实验

```
# unit: pure pricing function
# integration: repository + real test DB
# e2e: HTTP request -> DB -> response
```

易错点 / GOTCHAS

- 把 mock 数量当 unit test 质量指标。
- 追求覆盖率数字而忽略断言价值。

面试官连续追问

1. contract test 适合什么场景?
2. 数据库测试如何隔离数据?

> **举一反三** 测试策略优化的是反馈质量/速度，不是某一种测试数量最大化。

Q88 mock 应该 mock 什么?

难度：★★★ 频率：极高 关键词：mock, boundary, test design

30-60 秒面试标准回答

优先 mock 外部边界与不可控依赖（网络、时间、第三方服务），不要把被测模块内部每一步实现都 mock 掉。

原理拆解

- 过度 mock 让测试绑定实现细节，重构时大量无意义失败。
- patch 要发生在“被测代码查找对象的命名空间”。
- 对自有数据库/消息系统，很多关键行为更值得用轻量真实 integration test。

最小可运行实验

```
with patch('orders.service.payment_client.charge') as charge:
    charge.return_value = Receipt('ok')
    result = place_order(...)
    assert result.ok
```

易错点 / GOTCHAS

- patch 原定义模块而非使用位置。
- mock 返回结构与真实 SDK 不一致，测试产生虚假安全感。

面试官连续追问

1. fake/stub/mock/spies 的取舍?
2. 时间测试为什么适合注入 Clock?

> **举一反三** 最易测试的代码通常拥有显式边界，而不是最会用 mock 的代码。

Q89 virtual environment / lockfile 分别解决什么问题?

难度：★★ 频率：高 关键词：venv, dependencies, reproducibility

30-60 秒面试标准回答

virtual environment 隔离项目 Python 与安装包；lockfile/锁定策略解决“同一依赖声明在不同时间解析出不同版本”的可复现性问题。

原理拆解

- venv 隔离并不等于完全可复现，还要考虑 Python 版本、平台 wheel、系统库。
- requirements range 描述可接受集合，lock 描述一次确定解析结果。
- CI 应从干净环境验证安装和测试。

最小可运行实验

```
python -m venv .venv
# 激活后再由 pip/uv 等工具安装项目依赖
```

易错点 / GOTCHAS

- 把 venv 当容器/虚拟机级隔离。
- 只提交宽松 requirements，不固定生产构建解析结果。

面试官连续追问

1. editable install 是什么?
2. wheel 与源码构建如何影响可复现性?

> **举一反三** 依赖管理的目标是“可重复构建”，而不只是本机能 import。

Q90 pyproject.toml 在现代 Python 项目中的角色?

难度：★★★ 频率：高 关键词：pyproject.toml, packaging, build

30-60 秒面试标准回答

pyproject.toml 是现代 Python 项目的统一配置入口之一，可声明 build system、project metadata、dependencies，并承载众多工具配置。

原理拆解

- build frontend 与 build backend 是不同角色；PEP 517/518 把构建环境与后端协议标准化。
- wheel 是构建分发产物，sdist 是源分发；生产部署通常优先可复现 wheel。
- 工具配置集中能减少散落文件，但也要避免巨大不可维护配置。

最小可运行实验

```
[build-system]
requires = ['hatchling']
build-backend = 'hatchling.build'

[project]
name = 'myapp'
version = '0.1.0'
```

易错点 / GOTCHAS

- 把 pyproject.toml 仅理解成“替代 requirements.txt”。
- 混淆 build backend 与 package installer。

面试官连续追问

1. wheel vs sdist?
2. editable install 在 PEP 660 下如何工作?

> **举一反三** 工程岗应能解释从源码到 wheel 到安装环境的完整链路。

CHAPTER 10

性能优化、Web 后端与系统设计

把语言知识落到 profiling、内存、流处理、缓存和高并发 API 的综合场景。

本章目标：能够从 profile 数据出发做性能优化并完成 Python 服务系统设计。

本章题目

Q91 Python 程序慢，第一步该做什么？ 难度 ★★ · 频率 极高 · profiling, performance, measurement

Q92 list comprehension 为什么经常比显式 for 更快？ 难度 ★★ · 频率 中 · comprehension, bytecode, performance

Q93 为什么 NumPy 往往比 Python for loop 快很多？ 难度 ★★★ · 频率 极高 · NumPy, vectorization, memory layout

Q94 如何系统定位 Python memory leak？ 难度 ★★★★ · 频率 极高 · memory leak, tracemalloc, profiling

Q95 Django / Flask / FastAPI 应如何选择？ 难度 ★★★ · 频率 高 · web framework, Django, FastAPI

Q96 FastAPI async def endpoint 为什么仍可能很慢？ 难度 ★★★ · 频率 极高 · FastAPI, blocking, latency

Q97 手写 LRU Cache，为什么常用 HashMap + Doubly Linked List？ 难度 ★★★★ · 频率 极高 · LRU, hash map, linked list

Q98 100GB 日志不能进内存，如何处理？ 难度 ★★★ · 频率 极高 · streaming, external memory, large file

Q99 1 亿 URL 找 Top-K，如何设计？ 难度 ★★★★ · 频率 高 · Top-K, heap, partition

Q100 设计高并发 Python API，如何选择整体并发模型？ 难度 ★★★★★ · 频率 极高 · system design, API, concurrency

Q91 Python 程序慢，第一步该做什么？

难度：★★ 频率：极高 关键词：profiling, performance, measurement

30-60 秒面试标准回答

第一步是建立可重复基准并 profile，确认瓶颈属于 CPU、I/O、锁、数据库、网络、内存还是算法；不要先凭感觉重写。

原理拆解

- 宏观先看请求延迟分解/吞吐/CPU/IO，再用 cProfile、py-spy、scalene、数据库 tracing 等深入。
- 优化优先级通常是算法与 I/O 次数 > 数据结构 > 批处理/缓存 > 微优化。
- 性能变化必须用同一 workload 复测，避免优化基准之外的东西。

最小可运行实验

```
python -m cProfile -o profile.out app.py
# 生产环境可用采样 profiler，降低侵入性
```

易错点 / GOTCHAS

- 一开始就上 Cython/Rust。
- 只看平均延迟，不看 p95/p99 和吞吐。

面试官连续追问

1. sampling vs instrumentation profiler?
2. 如何构建 micro/macro benchmark?

> **举一反三** 性能工程是测量-假设-实验循环。

Q92 list comprehension 为什么经常比显式 for 更快?

难度：★★ 频率：中 关键词：comprehension, bytecode, performance

30-60 秒面试标准回答

简单 list comprehension 在 CPython 中通常有更紧凑的执行路径和更少的 Python 层方法查找/调度开销，但差异依赖版本与代码形态。

原理拆解

- 优势只在简单映射/过滤中明显；复杂逻辑可读性优先。
- Python 3.14 等版本持续优化解释器，旧 benchmark 不能永久代表未来。
- 如果结果不需要全部物化，generator expression 可能更省内存但未必更快。

最小可运行实验

```
result = [x * 2 for x in xs if x > 0]
# 清晰时使用；复杂分支请展开为普通循环
```

易错点 / GOTCHAS

- 把 comprehension 当无条件更快。
- 把复杂副作用塞入 comprehension。

面试官连续追问

1. map/filter 与 comprehension 如何比较?
2. 为什么性能结论要注明 Python 版本?

> **举一反三** 微优化必须建立在 profile 证据上。

Q93 为什么 NumPy 往往比 Python for loop 快很多?

难度：★★★ 频率：极高 关键词：NumPy, vectorization, memory layout

30-60 秒面试标准回答

NumPy 用紧凑同类型数组把循环下沉到优化的 C/Fortran/native kernels，减少 Python 对象、动态类型检查和解释器循环开销，并可利用 SIMD/BLAS。

原理拆解

- Python list 通常是对象引用数组，元素本身是独立 PyObject。
- vectorization 可能创建大临时数组，内存带宽成为瓶颈；不是所有表达式都应强行向量化。
- 底层 BLAS 自己多线程时，上层进程/线程并行要避免 oversubscription。

最小可运行实验

```
import numpy as np
a = np.arange(1_000_000, dtype=np.float64)
b = a * 2 + 1
```

易错点 / GOTCHAS

- 回答只有“因为 NumPy 用 C 写的”。
- 忽略 dtype、contiguity、temporary allocations。

面试官连续追问

1. broadcasting 为什么不等于复制?
2. memory bandwidth bound 怎么识别?

> **举一反三** 真正原因是数据布局 + 降低解释器开销 + 优化内核。

Q94 如何系统定位 Python memory leak?

难度：★★★★ 频率：极高 关键词：memory leak, tracemalloc, profiling

30-60 秒面试标准回答

先区分活对象持续增长、缓存/队列增长、allocator retained memory、native memory。再通过稳定 workload、快照 diff、对象引用链与系统指标建立证据。

原理拆解

- tracemalloc 能追踪许多 Python allocation traceback，但看不到所有 native allocation。
- gc/get_referrers/objgraph 可帮助追引用链，但会扰动程序，适合诊断环境。
- 缓存无界增长、Task/回调引用、全局 registry、队列消费不及生产是常见“逻辑泄漏”。

最小可运行实验

```
tracemalloc.start()
before = tracemalloc.take_snapshot()
run_workload()
after = tracemalloc.take_snapshot()
for stat in after.compare_to(before, 'lineno')[:20]:
    print(stat)
```

易错点 / GOTCHAS

- 只看 RSS。
- 生产直接大量调用 gc.get_objects 导致严重开销。

面试官连续追问

1. native extension leak 如何定位?
2. 如何判断缓存增长是预期还是泄漏?

> **举一反三** 定位泄漏的关键是“随相同 workload 哪类对象单调增长”。

Q95 Django / Flask / FastAPI 应如何选择?

难度：★★★ 频率：高 关键词：web framework, Django, FastAPI

30-60 秒面试标准回答

从产品需求与团队约束选择，而不是只比 benchmark：Django 提供完整 ORM/admin/auth 生态；FastAPI 偏 API、typing/ASGI/async 友好；Flask 更小更灵活。

原理拆解

- 框架吞吐常不是系统瓶颈，数据库、外部 API、序列化、业务逻辑更重要。
- 成熟度、招聘/运维经验、迁移成本和生态插件往往比纯请求/s benchmark 更关键。
- async framework 若底层依赖都是 sync blocking，收益有限。

最小可运行实验

```
# 选择问题先列约束：
# admin? ORM? async upstream? WebSocket? 团队经验? 插件生态? 部署模型?
```

易错点 / GOTCHAS

- “FastAPI 一定最快所以最好”。
- “Django 太重所以不能做高并发”。

面试官连续追问

1. ASGI vs WSGI?
2. Django async 支持边界?

> **举一反三** 面试把框架题回答成 trade-off 题，比站队更专业。

Q96 FastAPI async def endpoint 为什么仍可能很慢?

难度：★★★ 频率：极高 关键词：FastAPI, blocking, latency

30-60 秒面试标准回答

async def 只说明函数是 coroutine；内部如果调用 blocking DB/HTTP SDK、time.sleep、CPU-heavy 代码，仍会阻塞 event loop 或占满 worker。

原理拆解

- 要检查整条 dependency chain：路由、依赖注入、数据库驱动、序列化、日志、第三方 SDK。
- CPU-heavy validation/serialization 在高并发下也可能成为瓶颈。
- 连接池过小表现为排队，过大又可能压垮数据库。

最小可运行实验

```
@app.get('/x')
async def x():
    data = await async_client.get(...)  # 真正 async
    return data
```

易错点 / GOTCHAS

- 看到 async def 就断言 non-blocking。
- 无限调大 uvicorn workers 掩盖 DB 瓶颈。

面试官连续追问

1. 如何监控 event loop lag?
2. sync endpoint 在 FastAPI 中如何执行?

> **举一反三** 端到端延迟 = 排队 + 应用 + 下游；必须分层观测。

Q97 手写 LRU Cache，为什么常用 HashMap + Doubly Linked List?

难度：★★★★ 频率：极高 关键词：LRU, hash map, linked list

30-60 秒面试标准回答

HashMap 提供 O(1) key -> node 查找；双向链表提供 O(1) 删除任意节点并移动到 MRU 端，因此 get/put 可达平均 O(1)。

原理拆解

- 容量满时从 LRU 端淘汰。
- thread-safe LRU 需要保护 map/list 的联合不变量，不能只给其中一个结构加锁。
- 生产 cache 还要考虑 TTL、权重容量、stampede、negative cache、metrics。

最小可运行实验

```
# node: key, value, prev, next
# map[key] -> node
# get: lookup + move_to_front
# put: update/insert + evict_tail
```

易错点 / GOTCHAS

- 用 list 查 key 使 get 退化 O(n)。
- 移动节点时漏更新相邻 prev/next。

面试官连续追问

1. LFU 与 LRU 的 workload 差异?
2. 如何做 sharded lock 降低竞争?

> **举一反三** 算法题答完基本结构后，主动加“并发与缓存一致性”是工程岗加分项。

Q98 100GB 日志不能进内存，如何处理?

难度：★★★ 频率：极高 关键词：streaming, external memory, large file

30-60 秒面试标准回答

第一原则是流式读取并保持 bounded memory；具体聚合可用分块、hash partition、external sort、磁盘数据库或流处理系统。

原理拆解

- `for line in f` 会增量读取，不需要 `f.read()` 全部加载。
- 若 distinct key 太多，内存 hash map 仍会爆，需要 partition 到磁盘再局部聚合。
- 压缩文件、编码错误、部分行损坏、checkpoint/restart 都是生产数据管线要考虑的问题。

最小可运行实验

```
from collections import Counter
counts = Counter()
with open(path, encoding='utf-8', errors='replace') as f:
    for line in f:
        counts[extract_key(line)] += 1
```

易错点 / GOTCHAS

- 只把 `read()` 改成 `readlines()`，仍全部物化。
- 流式读取但把所有解析结果 append 到 list。

面试官连续追问

1. external sort 如何工作?
2. 什么时候用 DuckDB/SQLite 比手写更好?

> **举一反三** “内存有界”必须贯穿整个 pipeline，而不只是输入阶段。

Q99 1 亿 URL 找 Top-K，如何设计?

难度：★★★★ 频率：高 关键词：Top-K, heap, partition

30-60 秒面试标准回答

若 distinct URL 数可放内存：hash count O(n)，再用大小 K 的 heap 做 O(m log K)。若无法放内存，先按 hash 分区到磁盘/分布式节点，再局部计数和局部 Top-K，最后合并。

原理拆解

- 决定方案的关键参数是总记录数 n、distinct 数 m、内存、是否允许近似。
- 局部 Top-K 的并集包含全局 Top-K 候选，可再做一次 merge。
- 如果允许近似，可考虑 Count-Min Sketch + heavy hitters，但要明确误差。

最小可运行实验

```
# exact, memory allows
from collections import Counter
from heapq import nlargest
counts = Counter(stream)
print(nlargest(k, counts.items(), key=lambda kv: kv[1]))
```

易错点 / GOTCHAS

- 直接 `sorted(counts.items())` 忽略 m 很大时 O(m log m)。
- 只讲算法不问内存/是否允许近似。

面试官连续追问

1. 为什么 local Top-K 可以缩小 merge 候选?
2. Count-Min Sketch 的误差方向是什么?

> **举一反三** 系统设计题先问约束，才能判断 exact vs approximate、单机 vs distributed。

Q100 设计高并发 Python API，如何选择整体并发模型?

难度：★★★★★ 频率：极高 关键词：system design, API, concurrency

30-60 秒面试标准回答

先按请求关键路径拆成 I/O-bound 与 CPU-bound，再分别安排 async I/O、线程兼容层、进程/free-threaded/subinterpreter/独立计算 worker，并用连接池、队列、限流和 backpressure 控制资源。

原理拆解

- Web worker 负责轻量编排，不应无限承载重计算。
- 数据库/上游连接池、timeout、retry、circuit breaker、cache 和 queue 都是容量模型的一部分。
- 稳定性要用 p95/p99、event-loop lag、queue depth、pool saturation、error rate 等指标观测；扩 worker 前先找真正瓶颈。

最小可运行实验

```
# 一个常见分层
# Client -> API/ASGI workers -> async DB/HTTP
#                          -> bounded job queue -> CPU workers
#                          -> cache
# 全链路都有 timeout/backpressure/metrics
```

易错点 / GOTCHAS

- “FastAPI + async + 多 worker”作为完整答案。
- 所有下游 timeout 都用默认无限等待。
- CPU job 与 API worker 共用容量导致雪崩。

面试官连续追问

1. 如何做 graceful shutdown?
2. 如何计算连接池大小?
3. 如何避免 retry storm?
4. free-threaded 部署前要验证哪些第三方库?

> **举一反三** 这道题的评分点不在框架名，而在 workload 分类、容量隔离、失败治理和可观测性。

附录 A 面试冲刺路线

如果只有 3 天，不建议平均刷 100 题。下面按“出现频率 × 知识枢纽性”排序。

S 级：必须口述流畅

覆盖对象模型、容器、闭包、数据模型、CPython、并发、async 与系统设计。

Q03 Q04 Q09 Q14 Q15 Q20 Q24 Q25 Q26 Q35 Q37 Q51 Q61 Q63 Q64 Q73 Q75 Q76 Q91 Q100

A 级：中高级岗位高频

用于连续追问和工程深度。

Q16 Q30 Q34 Q38 Q40 Q54 Q57 Q58 Q62 Q66 Q68 Q70 Q72 Q74 Q77 Q78 Q80 Q83 Q85 Q94 Q96 Q97

B 级：补齐覆盖面

根据目标岗位选择性复习。

Q01 Q02 Q05 Q06 Q07 Q08 Q10 Q11 Q12 Q13 Q17 Q18 Q19 Q21 Q22 Q23 Q27 Q28 Q29 Q31 Q32 Q33 Q36 Q39 Q41 Q42 Q43 Q44 Q45 Q46 Q47 Q48 Q49 Q50 Q52 Q53 Q55 Q56 Q59 Q60 Q65 Q67 Q69 Q71 Q79 Q81 Q82 Q84 Q86 Q87 Q88 Q89 Q90 Q92 Q93 Q95 Q98 Q99

附录 B 常用复杂度速查

操作	结构	典型复杂度	备注
append / pop tail	list	摊还 O(1)	扩容时单次可 O(n)
insert/pop head	list	O(n)	需要移动引用
index access	list/tuple	O(1)	连续数组索引
membership	list/tuple	O(n)	线性扫描
lookup/insert	dict/set	平均 O(1)	依赖 hash/equality；最坏可退化
appendleft/popleft	deque	O(1)	两端队列优势
heap push/pop	heapq	O(log n)	Top-K 常用
sort	list.sort/sorted	O(n log n)	Timsort，近有序数据有优势

附录 C Python 并发模型决策表

场景	首选候选	优势	主要成本	版本备注
大量网络 I/O	asyncio	单线程高并发、结构化任务管理	要求全链路非阻塞	TaskGroup 3.11+
阻塞 I/O SDK	ThreadPool / to_thread	兼容旧库	线程数/共享状态	默认 CPython 仍受 GIL 影响 CPU
CPU-heavy pure Python	ProcessPool	成熟多核方案	pickle/IPC/内存	跨平台 start method 不同
CPU-heavy, free-threaded ready	Threads	共享内存 + 真并行	线程安全/扩展兼容	3.13+ 可选；3.14 非默认
隔离解释器并行	InterpreterPoolExecutor	各解释器独立 GIL、多核	数据隔离/序列化	Python 3.14 新增
重计算服务化	独立 worker/service	容量隔离、可水平扩展	系统复杂度	高并发 API 常用

附录 D 20 条高分面试原则

01. 先区分 Python 语言规范与 CPython 实现细节。

02. 复杂度一定注明平均、摊还还是最坏。

03. 参数传递用“对象共享/名字绑定”解释，不用含混的“引用传递”。

04. 可变性、哈希性、身份、相等是四个不同概念。

05. 默认参数与闭包题都要问“何时绑定/何时求值”。

06. descriptor 是 property、method binding、ORM field 的共同底层协议。

07. super 沿 MRO 继续查找，不等于固定调用直接父类。

08. generator 的价值是惰性和状态暂停，不等于异步。

09. 资源释放使用 context manager，不依赖 GC 时机。

10. RSS 不下降不自动等于 Python 对象泄漏。

11. GIL 不等于业务线程安全。

12. 2026 回答 GIL 必须补充 free-threaded build。

13. 并发模型选择先分类 workload，再看通信/隔离成本。

14. async def 不会把内部 blocking 代码自动变成非阻塞。

15. TaskGroup 的价值是任务所有权和失败/取消传播。

16. 取消是控制流，cleanup 后通常继续传播 CancelledError。

17. type hints 是静态契约，不是默认 runtime validation。

18. mock 边界而不是 mock 每个实现细节。

19. 性能优化先 profile，算法/I/O 次数通常优先于微优化。

20. 系统设计答案必须包含 backpressure、timeout、容量隔离和 observability。

参考资料与版本依据

本书以官方文档为版本敏感事实的最终依据。以下资料尤其用于并发、asyncio、Data Model 等章节。

[1] Python 3.14 concurrent.futures - InterpreterPoolExecutor: https://docs.python.org/3.14/library/concurrent.futures.html

[2] Python 3.14 asyncio - Coroutines and Tasks / TaskGroup / Cancellation: https://docs.python.org/3.14/library/asyncio-task.html

[3] Python 3.14 threading - GIL and performance considerations: https://docs.python.org/3.14/library/threading.html

[4] Python 3.14 Free-threading HOWTO: https://docs.python.org/3.14/howto/free-threading-python.html

[5] PEP 779 - Criteria for supported status for free-threaded Python: https://peps.python.org/pep-0779/

[6] Python 3.14 Data Model: https://docs.python.org/3.14/reference/datamodel.html

[7] Python typing documentation: https://docs.python.org/3/library/typing.html

[8] Python descriptor HOWTO: https://docs.python.org/3/howto/descriptor.html

[9] Python import system / importlib documentation: https://docs.python.org/3/reference/import.html

说明：本书借鉴题解型技术书的“题目驱动、层层追问、举一反三”组织方法，但内容、文字、示例与版式均为重新编写，不复制任何受版权保护作品的具体文本或页面设计。

100 题总索引

题号	题目	难度	频率
Q01	Python 是解释型语言还是编译型语言?	★★	高
Q02	is 和 == 有什么区别?	★	高
Q03	Python 变量保存的是值还是对象?	★★	高
Q04	Python 是 pass-by-value 还是 pass-by-reference?	★★	高
Q05	mutable 和 immutable 的本质区别是什么?	★★	高
Q06	None 到底是什么? 为什么推荐 is None?	★	高
Q07	Python 的 truthy / falsy 规则是什么?	★★	高
Q08	and / or 为什么不一定返回 bool?	★★	中
Q09	为什么 mutable default argument 是经典陷阱?	★★	极高
Q10	global 和 nonlocal 有什么区别? LEGB 如何工作?	★★	高
Q11	list 和 tuple 的区别是什么?	★	极高
Q12	list.append 为什么是摊还 O(1)?	★★	高
Q13	为什么 list.insert(0, x) / pop(0) 慢?	★	高
Q14	Python dict 查找为什么平均 O(1)?	★★★	极高
Q15	什么对象可以作为 dict key?	★★	极高
Q16	为什么实现 __eq__ 后必须认真考虑 __hash__?	★★★	高
Q17	dict 是否保证插入顺序?	★	高
Q18	dict.get、setdefault、defaultdict 如何选择?	★★	高
Q19	set 为什么能高效查重?	★	极高
Q20	浅拷贝和深拷贝有什么区	★★	极高

题号	题目	难度	频率
	别?		
Q21	*args 和 **kwargs 的本质是什么?	★	极高
Q22	positional-only 与 keyword-only 参数解决什么问题?	★★	中
Q23	为什么说函数是一等对象?	★	高
Q24	什么是闭包?	★★	极高
Q25	late binding closure 为什么会得到 [2,2,2]?	★★★	极高
Q26	装饰器本质是什么?	★★	极高
Q27	为什么 decorator 通常需要 functools.wraps?	★★	高
Q28	如何实现带参数装饰器?	★★	高
Q29	lambda 与普通 def 的区别是什么?	★	中
Q30	functools.lru_cache 有哪些工程陷阱?	★★★	高
Q31	__new__ 和 __init__ 有什么区别?	★★★	高
Q32	实例方法、classmethod、staticmethod 如何选择?	★★	极高
Q33	Python 的“private”是怎么实现的?	★★	高
Q34	__getattribute__ 和 __getattr__ 有什么区别?	★★★★	高
Q35	什么是 descriptor? 为什么它如此重要?	★★★★★	高
Q36	property 是如何工作的?	★★★	高
Q37	MRO 是什么? C3 linearization 解决什么问题?	★★★★	高
Q38	super() 为什么不是“调用父类”?	★★★★	高
Q39	什么是 metaclass? 什么时候真的需要?	★★★★★	中
Q40	__slots__ 有什么用? 有哪些	★★★★	高

题号	题目	难度	频率
	坑?		
Q41	iterable 和 iterator 有什么区别?	★★	极高
Q42	iter(x) 实际发生了什么?	★★★	高
Q43	generator 与普通函数有什么区别?	★★	极高
Q44	为什么 generator 通常更省内存?	★★	极高
Q45	generator.send() 有什么语义?	★★★★	中
Q46	yield from 不只是语法糖吗?	★★★★	中
Q47	为什么 generator/iterator 通常只能消费一次?	★★	高
Q48	context manager 的核心协议是什么?	★★	极高
Q49	with 为什么比手写 try/finally 更好?	★★	高
Q50	contextlib.contextmanager 是如何把 generator 变成上下文管理器的?	★★★	中
Q51	CPython 如何管理对象生命周期?	★★★★	极高
Q52	del x 是否等于删除对象?	★★	极高
Q53	什么是 reference cycle? 循环 GC 为什么需要它?	★★★	高
Q54	weakref 适合解决什么问题?	★★★	中
Q55	为什么对象释放后进程 RSS 不一定下降?	★★★★	中
Q56	sys.getsizeof 为什么不是对象“总内存”?	★★	高
Q57	Python import 的核心流程是什么?	★★★	高
Q58	circular import 为什么会产生 partially initialized module?	★★★	极高
Q59	.pyc 是什么? 什么时候会失	★★	中

题号	题目	难度	频率
	效?		
Q60	monkey patching 什么时候有用，为什么危险?	★★★	中
Q61	GIL 是什么？如何用现代版本回答?	★★★	极高
Q62	“有 GIL 所以不需要锁”为什么是错的?	★★★	极高
Q63	Python 3.14 free-threaded CPython 改变了什么?	★★★★★	高
Q64	Thread vs Process 应该怎么选?	★★★	极高
Q65	多进程的主要代价是什么?	★★	高
Q66	multiprocessing 为什么经常出现 pickle error?	★★★	高
Q67	ThreadPoolExecutor 与 ProcessPoolExecutor 的关键差异?	★★	极高
Q68	fork、spawn、forkserver 有什么区别?	★★★★	高
Q69	什么是 race condition？如何系统治理?	★★	极高
Q70	Python 3.14 InterpreterPoolExecutor 是什么?	★★★★★	高
Q71	concurrency 与 parallelism 有什么区别?	★	极高
Q72	coroutine、Task、Future 分别是什么?	★★★	极高
Q73	await 的核心语义是什么?	★★	极高
Q74	为什么 async 函数里 time.sleep 是严重问题?	★	极高
Q75	asyncio.gather 与 TaskGroup 怎么选?	★★★★	极高
Q76	为什么不能随便吞 asyncio.CancelledError?	★★★★	极高
Q77	async code 中如何调用阻塞	★★	极高

题号	题目	难度	频率
	函数?		
Q78	async 能解决 CPU-bound 吗?	★★	极高
Q79	Semaphore 在 async 服务中解决什么问题?	★★	高
Q80	如何设计一个生产级 asyncio worker?	★★★★★	极高
Q81	动态语言为什么仍需要 type hints?	★	极高
Q82	Any 和 object 有什么区别?	★★★	高
Q83	Protocol 解决了什么问题?	★★★★	高
Q84	TypeVar / 新泛型语法解决什么问题?	★★★★	高
Q85	ParamSpec 为什么适合给装饰器做 typing?	★★★★★	中
Q86	dataclass 解决了什么？有哪些陷阱?	★★	极高
Q87	unit test 与 integration test 如何划分?	★★	高
Q88	mock 应该 mock 什么?	★★★	极高
Q89	virtual environment / lockfile 分别解决什么问题?	★★	高
Q90	pyproject.toml 在现代 Python 项目中的角色?	★★★	高
Q91	Python 程序慢，第一步该做什么?	★★	极高
Q92	list comprehension 为什么经常比显式 for 更快?	★★	中
Q93	为什么 NumPy 往往比 Python for loop 快很多?	★★★	极高
Q94	如何系统定位 Python memory leak?	★★★★	极高
Q95	Django / Flask / FastAPI 应如何选择?	★★★	高
Q96	FastAPI async def endpoint 为什么仍可能很慢?	★★★	极高

题号	题目	难度	频率
Q97	手写 LRU Cache，为什么常用 HashMap + Doubly Linked List?	★★★★	极高
Q98	100GB 日志不能进内存，如何处理?	★★★	极高
Q99	1 亿 URL 找 Top-K，如何设计?	★★★★	高
Q100	设计高并发 Python API，如何选择整体并发模型?	★★★★★	极高